

Contents

Notation	5
Mathematical Background	7
Linear Algebra Foundations	7
Singular Value Decomposition (SVD)	7
Condition Number and Matrix Conditioning	7
Moore-Penrose Pseudoinverse	7
Trace Operator	8
Principal Component Analysis (PCA)	8
Problem Formulation	8
Optimization Objective	8
Explained Variance	9
Information Theory	9
Self-Information	9
Shannon Entropy	9
Kullback-Leibler Divergence	10
Cross-Entropy	10
Probabilistic Classification	10
Predictive Entropy	11
Calibration	11
Reliability Diagrams	13
Cross-Entropy as a Calibration Metric	14
Expected Calibration Error (ECE)	15
Confidence-Entropy Relationship	15
Information-Theoretic Interpretation	16
Optimization Theory	16
Stochastic Gradient Descent (SGD)	16
Momentum	17
Adam (Adaptive Moment Estimation)	17
Theory	18
Linear Classification: Softmax Regression	18
Model Definition	18
Loss Function	18
Gradient Computation	18

Geometric Interpretation	19
One-Hidden-Layer Neural Network	19
Model Definition	19
Universal Approximation Theorem	19
Forward Pass	22
Backward Pass (Backpropagation).....	22
Analytical calculation of backpropogation	23
Geometric Interpretation	24
Optimization Landscape and Capacity	24
Convex vs Non-Convex Optimization	24
Capacity and Generalization	25
Condition Number and Gradient Flow.....	26
Methods	26
Datasets and Preprocessing	26
Linear Gaussian Dataset	26
Moons Dataset	27
Digits Dataset	27
Data Centering	27
Model Implementations	28
Softmax Regression	28
One-Hidden-Layer Neural Network.....	28
Xavier/He Initialization	28
Training Protocol	30
Standard Training Loop	30
Hyperparameters.....	30
Model Selection.....	31
Evaluation Metrics.....	31
Accuracy	31
Cross-Entropy Loss.....	31
Confidence and Entropy.....	31
Repeated-Seed Evaluation	32
PCA Analysis	33
Implementation.....	33
Projection and Reconstruction	33
Classification on Reduced Dimensions	34

Numerical Stability Considerations.....	34
Softmax Stabilization	34
Gradient Checking	34
NaN/Inf Detection.....	34
Experimental Protocol	35
Core Experiments	35
Ablation Studies	35
Advanced Analysis.....	35
Reproducibility	35
Implementation Sanity Checks.....	36
Gradient Check	36
Probability Sum Check	36
NaN/Inf Detection	36
Loss Decrease on Tiny Subset.....	37
Experiments	37
Core Experiments.....	37
Linear Gaussian Task	37
Moons Task	38
Digits Benchmark	40
Required Ablations.....	43
Capacity Ablation (Moons Task).....	43
Optimizer Study (Digits Benchmark)	45
Failure Case Analysis: Under-Capacity Network.....	46
Advanced Analysis (Track A / Track B)	48
Track A: PCA/SVD Analysis	48
Additional 3D Visualization.....	51
Track B: Prediction Confidence and Reliability	53
Summary of Key Findings	57
Conclusion	60
Discussion	61
When is the Linear Model Geometrically Sufficient, and When is it Not?	61
What Representational Change Does the Hidden Layer Provide?.....	62
When Does Additional Model Complexity Fail to Justify Itself?	62
What Does Failure Case Teach About Optimization, Capacity, or Generalization?	63
What Do Repeated-Seed Statistics Add Beyond a Single Run?	64

The Practical Reality: Numerical Precision, Linear Algebra, and Information Theory	64
Interpretation of Results.....	66
Where Nonlinearity Helped and Why.....	66
Supported by Evidence vs. Remains Uncertain	67
Supported by Evidence	67
Remains Uncertain	67
Optimizer Study: What the Results Mean.....	68
Limitations	71
Architectural Limitations	71
Dataset Limitations	71
Hyperparameter Limitations	71
Evaluation Limitations	71
Reproducibility Limitations	72
Generalization Limitations	72
Summary: Linear vs Nonlinear, What We Know and What We Don't	74
What We Know (Evidence-Backed)	74
What Remains Uncertain	74
Central Question: When Does Nonlinearity Help?.....	76

**LOOK AT ALGEBRAIC/LINEAR ALGEBRA PROPERTIES OF PCA/SVD,
MATRICES ++**

THERE CAN BE A LITTLE DETAILED ANALYSIS OF DATASETS |- -|

Notation

Symbol	Definition
$x \in \mathbb{R}^d$	Input feature vector $d = 2$ for synthetic tasks, 64 for digits)
$y \in \{1, \dots, k\}$	Class label ($k = 2$ for synthetic, 10 for digits)
h	Number of hidden units in the neural network
$W_1 \in \mathbb{R}^{h \times d}$	Input-to-hidden weight matrix
$b_1 \in \mathbb{R}^h$	Hidden layer bias
$W_2 \in \mathbb{R}^{k \times h}$	Hidden-to-output weight matrix
$b_2 \in \mathbb{R}^k$	Output layer bias
$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Hyperbolic tangent activation function
$\text{softmax}(s_j) = \frac{e^{s_j}}{\sum_l e^{s_l}}$	Softmax activation for probability outputs
θ	Model parameters (weights and biases)
ϵ	Learning rate
α	Momentum coefficient (typically 0.9)
$g_t = \nabla_{\theta} L(\theta_t)$	Gradient of loss at time step t
$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$	Condition number of a matrix (ratio of largest to smallest eigenvalue)
$P(y = c \mid x)$	Probability that input x belongs to class c . Given x , the probability of class label being c .
$C(x) = \max_c P(y = c \mid x)$	Prediction confidence (maximum predicted probability)
$H(P) = - \sum_{c=1}^k p_c \ln p_c$	Shannon entropy of a probability distribution (predictive entropy)
$I(x) = -\ln P(x)$	Self-information (surprisal) of an event
$D_{KL}(P \parallel Q) = \sum_x P(x) \ln \frac{P(x)}{Q(x)}$	Kullback-Leibler divergence between distributions P and Q
$H(P, Q) = - \sum_x P(x) \ln Q(x)$	Cross-entropy between true distribution P and model distribution Q

$\text{NLL} = -\frac{1}{n} \sum_{i=1}^n \ln P(y = y_i x_i)$	Negative log-likelihood (mean cross-entropy loss)
$\mathcal{L}(\theta) = \text{NLL} + \frac{\lambda}{2} \ \theta\ _2^2$	Regularized loss function (cross-entropy + L2 regularization)
$\hat{y}_i = \arg \max_c P(y = c x_i)$	Predicted class label
$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}[\hat{y}_i = y_i]$	Classification accuracy
λ	L2 regularization coefficient $\lambda = 10^{-4}$ in experiments)
n	Number of samples in dataset or batch
B	Minibatch of samples
v_t	Velocity term in momentum optimizer
m_t	First moment estimate in Adam optimizer
v_t	Second moment estimate in Adam optimizer
β_1, β_2	Exponential decay rates for Adam (typically 0.9 and 0.999)
δ	Small constant for numerical stability in Adam ($\delta = 10^{-8}$)
$U \Sigma V^\top$	Singular Value Decomposition of matrix A
σ_i	Singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$
$A^+ = V \Sigma^+ U^\top$	Moore-Penrose pseudoinverse
$\text{Tr}(A) = \sum_i A_{ii}$	Trace of a square matrix
$\text{ECE} = \sum_{m=1}^M \frac{ B_m }{n} \text{acc}(B_m) - \text{conf}(B_m) $	Expected Calibration Error (calibration metric)

Mathematical Background

Linear Algebra Foundations

Singular Value Decomposition (SVD)

Every real matrix $A \in \mathbb{R}^{m \times n}$ admits a Singular Value Decomposition:

$$A = U\Sigma V^T$$

where:

- $U \in \mathbb{R}^{m \times m}$ is an orthogonal matrix whose columns are the left singular vectors
- $V \in \mathbb{R}^{n \times n}$ is an orthogonal matrix whose columns are the right singular vectors
- $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix containing the singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$

The singular values are the square roots of the eigenvalues of $A^T A$ and AA^T . SVD is numerically more stable than eigendecomposition because it does not require A to be square or symmetric.

Condition Number and Matrix Conditioning

The condition number of a matrix A is defined as:

$$\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} = \sqrt{\frac{\lambda_{\max}(A^T A)}{\lambda_{\min}(A^T A)}}$$

$\kappa(A)$ is large, the matrix is said to be ill-conditioned. This means:

- Small perturbations in input produce large changes in output
- Numerical errors are amplified during inversion or solving linear systems
- In neural networks, ill-conditioned weight matrices cause gradient explosion or vanishing

Moore-Penrose Pseudoinverse

For matrices that are not invertible (non-square or singular), the pseudoinverse provides the best approximate solution to $Ax = y$:

$$A^+ = V\Sigma^+U^T$$

where Σ^+ is obtained by taking the reciprocal of non-zero singular values and transposing. For overdetermined systems ($m > n$), A^+ gives the least-squares solution minimizing $\|Ax - y\|_2$. For underdetermined systems ($m < n$), it gives the minimum-norm solution minimizing $\|x\|_2$.

Trace Operator

The trace of a square matrix is the sum of its diagonal entries:

$$\text{Tr}(A) = \sum_{i=1}^n A_{ii}$$

The trace has the important property of cyclic invariance:

$$\text{Tr}(ABC) = \text{Tr}(BCA) = \text{Tr}(CAB)$$

This property is essential for deriving gradient expressions in matrix calculus, allowing us to rearrange terms without changing the result.

Principal Component Analysis (PCA)

Problem Formulation

Given centered data $X \in \mathbb{R}^{m \times d}$ (each column has zero mean), PCA finds the k -dimensional linear projection that maximizes the variance of the projected data.

Let $V_k \in \mathbb{R}^{d \times k}$ be a matrix with orthonormal columns. The projected data is $Z = XV_k$, with covariance

$$\frac{1}{n} Z^T Z = \frac{1}{n} V_k^T X^T X V_k$$

Optimization Objective

The optimal projection maximizes the trace of the projected covariance:

$$\max_{V_k^T V_k = I} \text{Tr}(V_k^T X^T X V_k)$$

The solution is given by the eigenvectors of $X^T X$ corresponding to the k largest eigenvalues. Equivalently, using SVD:

$$X = U \Sigma V^T$$

The columns of V (right singular vectors) are the principal components, and the projected data is $Z = U_k \Sigma_k$, where U_k and Σ_k contain the first k singular vectors and values.

Explained Variance

The variance captured by the i -th principal component is:

$$\lambda_i = \frac{\sigma_i^2}{n - 1}$$

The explained variance ratio for component i is:

$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^d \lambda_j}$$

Cumulative explained variance up to k components is:

$$\text{CVE}(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^d \lambda_j}$$

Information Theory

Self-Information

For an event x with probability $P(x)$, the self-information (or surprisal) is:

$$I(x) = -\ln P(x)$$

This measures the amount of information gained when the event occurs. Rare events ($P(x) \rightarrow 0$) have high information; common events ($P(x) \rightarrow 1$) have low information. When using natural logarithm, the unit is **nats**; using base-2 logarithm gives **bits** (Shannons).

Shannon Entropy

The entropy of a discrete random variable X with distribution P is the expected self-information:

$$H(P) = \mathbb{E}_{x \sim P}[-\ln P(x)] = -\sum_x P(x) \ln P(x)$$

Entropy measures the average uncertainty or unpredictability of the distribution:

- Low entropy: Distribution is concentrated (deterministic)
- High entropy: Distribution is spread out (uniform)

Kullback-Leibler Divergence

The KL divergence measures how one probability distribution Q diverges from a reference distribution P :

$$D_{KL}(P|Q) = \mathbb{E}_{\mathbf{x} \sim \mathbb{P}} \left[\ln \frac{P(\mathbf{x})}{Q(\mathbf{x})} \right] = \sum_{\mathbf{x}} P(\mathbf{x}) \ln \frac{P(\mathbf{x})}{Q(\mathbf{x})}$$

Properties:

- $D_{KL}(P|Q) \geq 0$, with equality if and only if $P = Q$
- Asymmetric: $D_{KL}(P|Q) \neq D_{KL}(Q|P)$
- Measures the extra information needed to encode samples from P using a code optimized for Q

Cross-Entropy

The cross-entropy between distributions P and Q is:

$$H(P, Q) = -\mathbb{E}_{\mathbf{x} \sim \mathbb{P}} \ln Q(\mathbf{x}) = H(P) + D_{KL}(P|Q)$$

In machine learning, P is the true distribution (often empirical from data) and Q is the model's predicted distribution. Since $H(P)$ is constant for a fixed dataset, minimizing cross-entropy is equivalent to minimizing $D_{KL}(P|Q)$.

Probabilistic Classification

For a classification problem with k classes, a probabilistic classifier outputs a probability distribution over classes for each input x :

$$P(y = c | x) = p_c, \quad \sum_{c=1}^k p_c = 1, \quad p_c \geq 0$$

The predicted class is $\hat{y} = \arg \max_c p_c$, and the associated confidence is:

$$C(x) = \max_c p_c$$

Confidence measures how certain the model is about its prediction. A confidence of 1.0 indicates absolute certainty, while a value near 1/k indicates near-uniform uncertainty.

Predictive Entropy

Uncertainty can be quantified using the Shannon entropy of the predicted distribution:

$$H(P) = - \sum_{c=1}^k p_c \ln p_c$$

Properties:

- $H(P) = 0$ when the distribution is deterministic (one class has probability 1)
- $H(P) = \ln k$ when the distribution is uniform $p_c = 1/k$ for all c
- Higher entropy indicates greater uncertainty; lower entropy indicates higher confidence

Unlike confidence, which only considers the maximum probability, entropy captures the full shape of the distribution. A model may have high confidence but still be wrong; entropy helps detect such cases by revealing when probabilities are spread across multiple classes.

Calibration

A probabilistic classifier is said to be calibrated if the predicted confidence matches the true accuracy for that confidence level. Formally, for all confidence thresholds $t \in [0,1]$:

$$\mathbb{P}(y = \hat{y} \mid \mathcal{C}(x) = t) = t$$

In practice, calibration is assessed by grouping predictions into bins by confidence and comparing the mean confidence within each bin to the empirical accuracy.

Let B_m be the set of predictions falling into bin m (e.g., confidence in $(0.8, 1.0]$). For each bin:

- Mean confidence:

$$\text{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathcal{C}(x_i)$$

- Empirical accuracy:

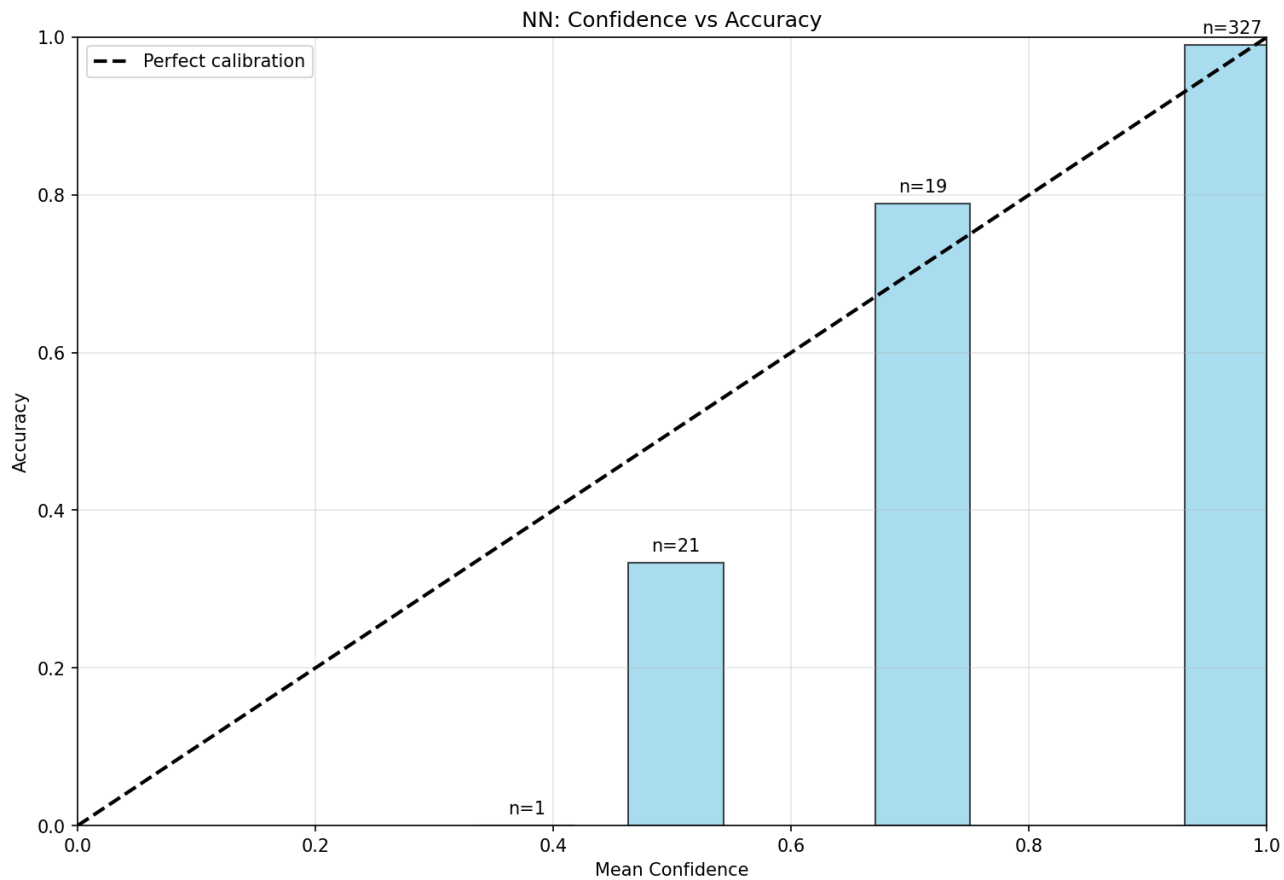
$$\text{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbb{1}[\hat{y}_i = y_i]$$

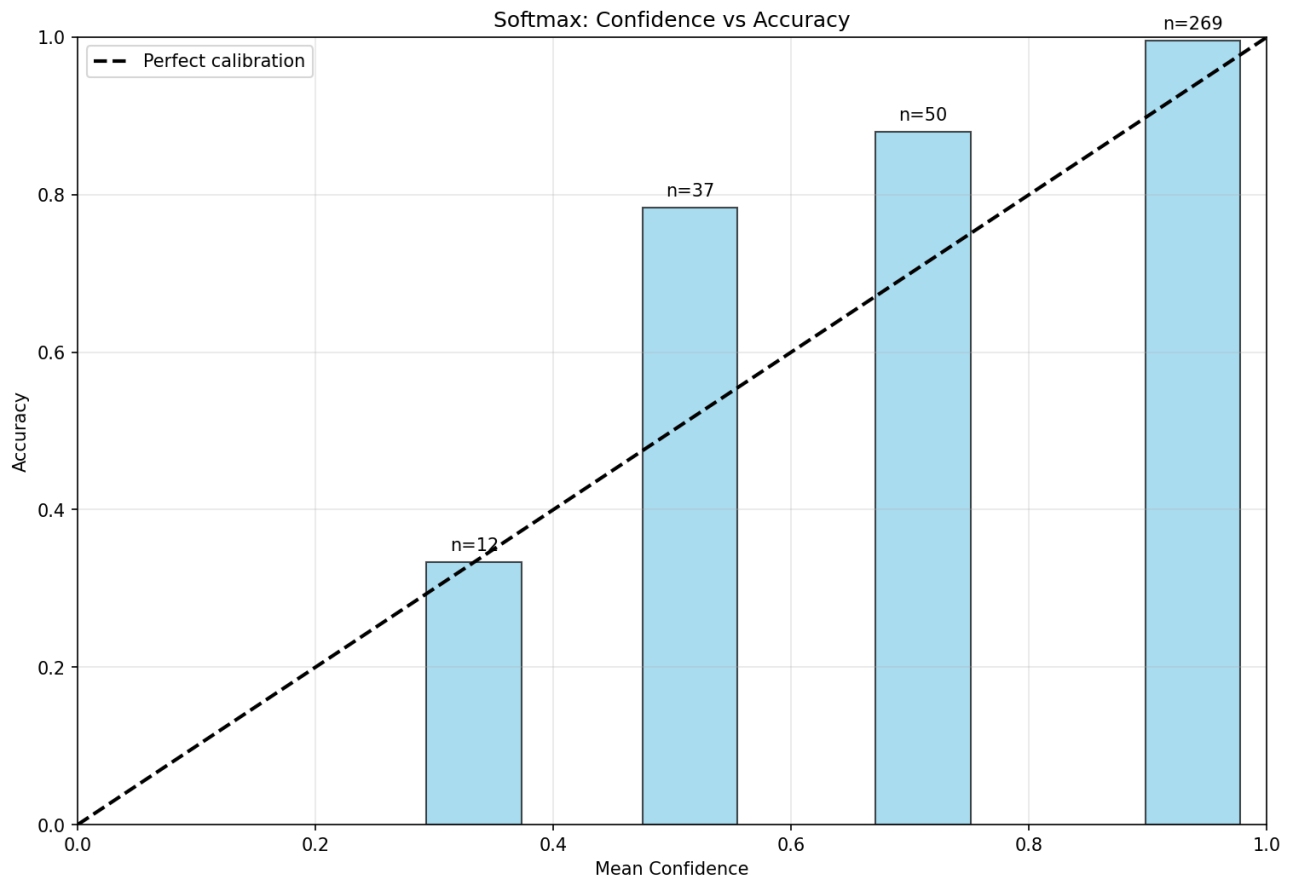
A perfectly calibrated model satisfies $\text{acc}(B_m) = \text{conf}(B_m)$ for all bins.

A little note here.

Reliability Diagrams

A reliability diagram (or calibration plot) visualizes calibration by plotting relative frequency in that predicted samples against mean confidence for each bin. Points above the diagonal indicate under-confidence (model is too conservative), while points below indicate over-confidence. A perfectly calibrated model lies on the diagonal $y = x$.





When choosing between different models, we need to consider both their accuracy and calibration quality. A model with slightly lower accuracy but better calibration might be more valuable than a highly accurate model with poor probability estimates.

By understanding calibration and its importance, we can build more reliable machine learning systems that users can trust not just for their predictions, but also for their confidence in those predictions.

A little note. Accuracy should be correctly interpreted as the relative frequency in that bin. If the model says that it is 0.20 confident (predict_proba=0.20), then we should expect that out of these samples, 20 percent of them are predicted class.

Cross-Entropy as a Calibration Metric

The cross-entropy loss, which we minimize during training, is connected to calibration:

$$L = -\frac{1}{n} \sum_{i=1}^n \ln P(y = y_i | x_i)$$

For a fixed accuracy, lower cross-entropy implies better calibration. This is because cross-entropy penalizes over-confident wrong predictions more severely than under-confident correct ones.

The relationship can be decomposed using the KL divergence:

$$H(P, Q) = H(P) + D_{KL}(P|Q)$$

where P is the true distribution (empirical from data), Q is the model's predicted distribution, and $D_{KL}(P|Q) \geq 0$ measures how far the model's distribution diverges from the truth. Minimizing cross-entropy is equivalent to minimizing this divergence, which encourages better calibration.

Expected Calibration Error (ECE)

A common scalar metric for calibration is the Expected Calibration Error:

$$ECE = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|$$

where:

- M is the number of bins
- $|B_m|$ is the number of samples in bin m
- n is the total number of samples

ECE measures the weighted average deviation between confidence and accuracy. Lower ECE indicates better calibration.

Confidence-Entropy Relationship

For a given prediction, there is a monotonic relationship between confidence and entropy. When the predicted distribution is concentrated (high confidence), entropy is low. When the distribution is spread out (low confidence), entropy is high. However, the relationship is not one-to-one: two different distributions can have the same confidence but different entropy values.

For a binary classifier with predicted probability p :

- Confidence:

$$C = \max(p, 1 - p)$$

- Entropy:

$$H = -p \ln p - (1 - p) \ln(1 - p)$$

This relationship generalizes to multi-class settings, where entropy provides a more nuanced view of uncertainty than confidence alone.

Information-Theoretic Interpretation

From an information theory perspective, a well-calibrated model assigns probabilities that accurately reflect the true underlying uncertainty. The KL divergence between the true distribution and the model's predictions:

$$D_{KL}(P_{\text{true}}|Q_{\text{model}}) = \sum_x P_{\text{true}}(x) \ln \frac{P_{\text{true}}(x)}{Q_{\text{model}}(x)}$$

measures the information loss when using the model's distribution to approximate the true distribution. A calibrated model minimizes this divergence, ensuring that its confidence levels are truthful representations of uncertainty.

This mathematical framework underpins the analysis of prediction confidence and reliability, allowing us to assess not just whether a model makes correct predictions, but whether its probability estimates are trustworthy.

Optimization Theory

Stochastic Gradient Descent (SGD)

SGD updates parameters using a minibatch/individual sample estimate of the gradient:

$$\theta_{t+1} = \theta_t - \epsilon g_t$$

Where:

$$g_t = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} L(\theta_t, x_i, y_i)$$

is the gradient over minibatch B ($B=1$, “really” stochastic case), and ϵ is the learning rate.

Limitations:

- Sensitive to learning rate selection
- Can oscillate in narrow valleys (poorly conditioned Hessian)
- May stall on plateaus where gradients are small

Momentum

Momentum introduces a velocity term that accumulates past gradients:

$$v_{t+1} = \alpha v_t - \epsilon g_t$$

$$\theta_{t+1} = \theta_t + v_{t+1}$$

where $\alpha \in [0,1)$ is the momentum coefficient (typically 0.9). This dampens oscillations and accelerates progress in consistent directions, enabling the optimizer to "roll over" small local minima and flat plateaus.

Adam (Adaptive Moment Estimation)

Adam combines momentum with per-parameter adaptive learning rates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{first moment})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{second moment})$$

$$\widehat{m}_t = m_t / (1 - \beta_1^t), \quad \widehat{v}_t = v_t / (1 - \beta_2^t) \quad (\text{bias correction})$$

$$\theta_{t+1} = \theta_t - \epsilon \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \delta}$$

where $\beta_1, \beta_2 \in [0,1)$ are decay rates (typically $\beta_1 = 0.9$, $\beta_2 = 0.999$), and δ is a small constant for numerical stability (typically 10^{-8}).

Adam's advantages:

- Per-parameter learning rates adapt to gradient scale
- Bias correction ensures stable initial steps
- Works well across diverse tasks with minimal tuning

Theory

Linear Classification: Softmax Regression

Model Definition

Softmax regression (multinomial logistic regression) defines the probability that input $x \in \mathbb{R}^d$ belongs to class $c \in \{1, \dots, k\}$ as:

$$P(y = c | x) = \frac{\exp(w_c^\top x + b_c)}{\sum_{j=1}^k \exp(w_j^\top x + b_j)}$$

where $w_c \in \mathbb{R}^d$ is the weight vector for class c and $b_c \in \mathbb{R}$ is the bias.

In matrix form, with $W \in \mathbb{R}^{k \times d}$ and $b \in \mathbb{R}^k$:

$$P = \text{softmax}(Wx + b)$$

Loss Function

For a dataset $\{(x_i, y_i)\}_{i=1}^n$ with one-hot encoded labels $Y \in \{0,1\}^{n \times k}$, the cross-entropy loss with L2 regularization is:

$$L(W, b) = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^k Y_{ic} \ln P_{ic} + \frac{\lambda}{2} (|W|_F^2 + |b|_2^2)$$

where λ is the regularization strength and $|W|_F^2 = \sum_{i,j} W_{ij}^2$ is the Frobenius norm squared.

Gradient Computation

The gradient of the loss with respect to the logits $S = Wx + b$ is:

$$\frac{\partial L}{\partial S} = \frac{1}{n} (P - Y)$$

This simplification arises because the derivative of cross-entropy with softmax yields the difference between predicted and true probabilities.

By the chain rule:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial S}^\top X = \frac{1}{n} (P - Y)^\top X + \lambda W$$

$$\frac{\partial L}{\partial b} = \sum_{i=1}^n \frac{\partial L}{\partial S_i} = \frac{1}{n} \sum_{i=1}^n (P_i - Y_i) + \lambda b$$

Geometric Interpretation

The decision boundary between classes c and c' is defined by:

$$(w_c - w_{c'})^T x + (b_c - b_{c'}) = 0$$

This is a hyperplane. The model partitions the input space into convex polytopes, each corresponding to a class. When the true decision boundary is curved, softmax must approximate it with a flat surface, inevitably leaving residual error.

One-Hidden-Layer Neural Network

Model Definition

A neural network with one hidden layer and $\tanh$ activation computes:

$$h = \tanh(W_1 x + b_1)$$

$$f(x) = \text{softmax}(W_2 h + b_2)$$

where:

- $W_1 \in \mathbb{R}^{h \times d}$ and $b_1 \in \mathbb{R}^h$ are hidden layer weights and biases
- $W_2 \in \mathbb{R}^{k \times h}$ and $b_2 \in \mathbb{R}^k$ are output layer weights and biases
- $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ is the hyperbolic tangent activation

Universal Approximation Theorem

The Universal Approximation Theorem is a foundational pillar of neural network theory. It states that a feedforward neural network with a single hidden layer and a finite number of neurons can approximate any continuous function on a compact subset of real numbers ($\mathbb{R}^m$), provided it uses an appropriate non-linear activation function.

Formal Definition

Let $C(K)$ be the space of continuous functions on a compact set $K \subseteq \mathbb{R}^m$. For any continuous function $f \in C(K)$ and any arbitrary error margin $\epsilon > 0$, there exists a feedforward neural network $\hat{f}$ with a single hidden layer such that:

$$|f(x) - \hat{f}(x)| < \epsilon \quad \text{for all } x \in K$$

This implies that with a sufficient number of neurons, a network can achieve any desired degree of accuracy.

Mathematical Structure

A neural network with a single hidden layer can be expressed as a composition of linear transformations and non-linear activations:

$$\hat{f}(x) = \sum_{i=1}^M c_i \cdot \sigma(w_i^T x + b_i)$$

Where:

- M : The number of neurons in the hidden layer.
- c_i : The weights of the output layer.
- w_i and b_i : The weights and biases of the hidden neurons.
- σ : The non-linear activation function (e.g., ReLU, Sigmoid, or Tanh).

The theorem relies on the fact that σ must be non-constant, bounded, and monotonically increasing. Without this non-linearity, the entire network would collapse into a simple linear regression model, which cannot approximate complex, curved patterns.

How the Approximation Works (The "Bump" Logic)

The proof of the theorem generally follows a three-step constructive logic:

1. Step Functions: Any continuous function can be represented as a series of tiny "steps" (rectangles).
2. Neural "Steps": By specifically tuning the weights and biases (w and b), a single neuron can be made to look like a step function. When you add two of these neurons together with opposite signs, they create a "bump."
3. Summation: By summing thousands of these "bumps" (each controlled by a neuron), the network can "carve out" the shape of any function, no matter how jagged or complex.

Theoretical Insights vs. Practical Reality

While the theorem proves that a solution **exists**, it provides a "existence proof" rather than a manual for success.

Insight	Practical Limitation
Expressiveness: Networks can model any relationship.	Efficiency: The number of neurons required might be billions or trillions for a single layer.
Simplicity: Only one hidden layer is mathematically "needed."	Depth: In practice, Deep networks (many thin layers) are much more efficient than Wide networks (one giant layer).
Convergence: A perfect weight configuration exists.	Optimization: Gradient descent might get stuck in local minima before finding those "perfect" weights.

The Universal Approximation Theorem guarantees the expressive power of neural networks. It tells us that if a model is failing to learn a pattern, the problem isn't that the network is "incapable" of representing the solution, the issue likely lies in the training data, the optimization process, or the depth of the architecture. This assurance is what justifies the move toward increasingly complex Deep Learning models.

Forward Pass

The forward pass computes all intermediate values:

$$Z_1 = XW_1^\top + b_1 \quad (\text{pre-activations})$$

$$H = \tanh(Z_1) \quad (\text{activations})$$

$$S = HW_2^\top + b_2 \quad (\text{output logits})$$

$$P = \text{softmax}(S) \quad (\text{probabilities})$$

Backward Pass (Backpropagation)

Backpropagation computes gradients using the chain rule. Starting from the loss:

Output layer sensitivity:

$$\frac{\partial L}{\partial S} = \frac{1}{n} (P - Y)$$

Output layer gradients:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial S}^\top H + \lambda W_2$$

$$\frac{\partial L}{\partial b_2} = \sum_{i=1}^n \frac{\partial L}{\partial S_i} + \lambda b_2$$

Hidden layer sensitivity: The gradient flows back through W_2 and the $\tanh$ derivative:

$$\frac{\partial L}{\partial H} = \frac{\partial L}{\partial S} W_2$$

$$\frac{\partial L}{\partial Z_1} = \frac{\partial L}{\partial H} \odot \tanh'(H)$$

here $\tanh'(H) = 1 - H^2$ element-wise), and $\odot$ denotes element-wise multiplication.

Hidden layer gradients:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial Z_1}^\top X + \lambda W_1$$

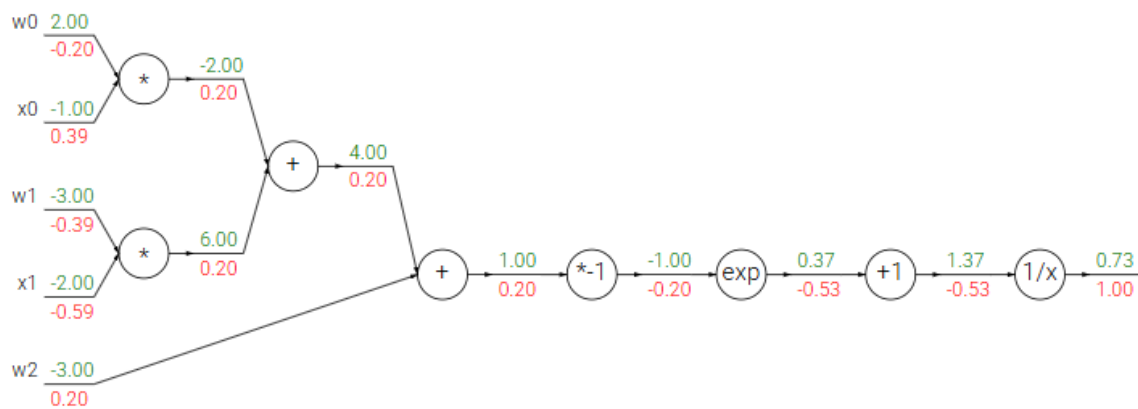
$$\frac{\partial L}{\partial b_1} = \sum_{i=1}^n \frac{\partial L}{\partial Z_{1i}} + \lambda b_1$$

Analytical calculation of backpropagation

In modern deep learning, the process of training a neural network relies on a sophisticated "memory-for-speed" trade-off. This mechanism, known as **Automatic Differentiation**, is the engine under the hood of almost every major machine learning framework.

The Forward Pass: The "Recording" Phase

When data moves through a model, the framework doesn't just calculate the final output. It simultaneously builds a **Computational Graph** (of every mathematical operation performed).



Example circuit for a 2D neuron with a sigmoid activation function. The inputs are $[x_0, x_1]$ and the (learnable) weights of the neuron are $[w_0, w_1, w_2]$. As we will see later, the neuron computes a dot product with the input and then its activation is softly squashed by the sigmoid function to be in range from 0 to 1.

As each layer computes its result, the framework **caches (stores)** the intermediate values, often called **activations**. These are not just extra data; they are the specific "snapshots" of the data as it existed at each step of the process.

Why We Can't Just Delete the Data

The reason frameworks hold onto these activations is the **Chain Rule** of calculus. To calculate how much a specific weight should change (the gradient), the math requires two things:

1. The error signal coming from the layer above.
2. The **original input** that passed through the layer during the forward pass.

If the framework deleted those inputs immediately to save memory, it would have no way to complete the multiplication required for backpropagation. It would be forced to re-calculate the entire forward pass from scratch for every single layer, making training impossibly slow.

The Backward Pass: The "Replay" Phase

During backpropagation, the framework traverses the computational graph in reverse.

- **Gradient Calculation:** It uses the cached activations to solve the derivative equations.
- **Memory Release:** As soon as the gradients for a specific layer are finalized and passed backward, the framework "frees" the cached activations for that layer.
- **Optimization:** Techniques like **Broadcasting** allow the framework to handle tensors of different sizes (like adding a single bias value to a whole batch of data) without manually duplicating data, while **Reduction** operations sum up individual errors into a single update value.

Efficiency vs. Flexibility

Different frameworks handle this "graph" in two primary ways:

- **Static Graphs:** The entire roadmap is defined and optimized before any data moves. This allows the system to pre-allocate memory very efficiently.
- **Dynamic Graphs:** The roadmap is built on the fly, layer by layer. This offers immense flexibility for complex models (like those with variable loops) but requires more active memory management.

Geometric Interpretation

The hidden layer creates a new representation $h(x) = \tanh(W_1 x + b_1)$. In this representation space, classes that were interleaved in the original space can become linearly separable. The final layer then applies a linear classifier in this transformed space. The decision boundary in input space is the preimage of a hyperplane through the $\tanh$ nonlinearity, enabling curved boundaries.

Optimization Landscape and Capacity

Convex vs Non-Convex Optimization

Softmax regression has a convex loss landscape because:

- The cross-entropy loss is convex in the parameters
- L2 regularization preserves convexity
- The problem has a unique global optimum

Neural networks have non-convex loss landscapes because:

- The composition of nonlinearities creates multiple local minima
- Saddle points vastly outnumber local minima in high dimensions
- Different random initializations converge to different solutions

Capacity and Generalization

The capacity of a model is its ability to represent complex functions. For one-hidden-layer networks, capacity grows with the number of hidden units h .

However:

- Underfitting: When h is too small, the model cannot represent the true function, leading to high bias
- Overfitting: When h is too large, the model can memorize noise, leading to high variance
- Optimal capacity: The "just right" h matches the intrinsic complexity of the data

Condition Number and Gradient Flow

The condition number of W_1 affects gradient propagation:

$$\kappa(W_1) = \frac{\sigma_{\max}(W_1)}{\sigma_{\min}(W_1)}$$

κ is large:

- Gradients become ill-conditioned
- Some directions are amplified, others vanish
- Optimization becomes slow and unstable

This explains why very narrow layers ($h = 1$, $h = 2$) fail: their weight matrices are nearly singular, making reliable gradient transmission impossible.

Methods

Datasets and Preprocessing

Linear Gaussian Dataset

Synthetic dataset of two Gaussian blobs with equal covariance matrices:

$$X \sim \mathcal{N}(\mu_0, \Sigma) \quad \text{for class 0}$$

$$X \sim \mathcal{N}(\mu_1, \Sigma) \quad \text{for class 1}$$

Parameters:

$$- \mu_0 = (-1, 0), \mu_1 = (1, 0)$$

$$- \Sigma = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}$$

$$- n_{\text{train}} = 1000, n_{\text{val}} = 200, n_{\text{test}} = 200$$

The dataset is linearly separable by the hyperplane $x_1 = 0$.

Moons Dataset

Synthetic dataset of two interleaving half-circles generated by:

$$x = (r \cos \theta + \delta_x r \sin \theta + \delta_y)$$

where:

- $r \in [0.5, 1.0]$ for class 0, $r \in [0.5, 1.0] +$ for class 1
- $\theta \in [0, \pi]$ for both classes
- $\delta_x, \delta_y \sim \mathcal{N}(0, 0.05)$ (noise)
- $n_{\text{train}} = 1000, n_{\text{val}} = 200, n_{\text{test}} = 200$

The optimal decision boundary is curved.

Digits Dataset

MNIST-like dataset of handwritten digits (0-9), preprocessed as:

- 8×8 grayscale images (64 pixels)
- Fixed train/val/test split: 1074/355/368 samples
- No additional preprocessing beyond centering for PCA analysis

THERE CAN BE A LITTLE DETAILED ANALYSIS OF DATASETS

Data Centering

For PCA analysis, we center the data needs to be centered:

$$X_{\text{centered}} = X -$$

where $\mu_j = \frac{1}{n} \sum_{i=1}^n X_{ij}$ is the mean of feature j . Centering is essential because PCA finds directions of maximum variance around the origin; without centering, components are biased toward the mean.

Model Implementations

Softmax Regression

Implemented with:

- Parameter initialization: $W \sim \mathcal{N}(0, 0.01^2)$, $b = 0$
- Forward pass: logits = $XW^T + b$, probabilities = softmax(logits)
- Loss: cross-entropy + L2 regularization with $\lambda = 10^{-4}$
- Optimization: SGD with learning rate $\epsilon = 0.05$, batch size 64

One-Hidden-Layer Neural Network

Implemented with:

- Parameter initialization: Xavier/He initialization:

$$W_1 \sim \mathcal{N}\left(0, \sqrt{\frac{2}{d}}\right), \quad W_2 \sim \mathcal{N}\left(0, \sqrt{\frac{2}{h}}\right)$$
$$b_1 = 0, \quad b_2 = 0$$

- Activation: *tanh* for hidden layer
- Forward pass:

$$Z_1 = XW_1^T + b_1,$$

$$H = \tanh(Z_1),$$

$$S = HW_2^T + b_2,$$

$$P = \text{softmax}(S)$$

- Loss: cross-entropy + L2 regularization with $\lambda = 10^{-4}$ applied to all parameters
- Optimization: SGD with learning rate $\epsilon = 0.05$, batch size 64

Xavier/He Initialization

Why Xavier/He?

Weight Initialization: Xavier and He Strategies

In deep learning, **weight initialization** is the process of setting the initial values for a neural network's weights. Because a network cannot begin training with empty or zeroed parameters (which would

prevent the model from "breaking symmetry" and learning), we must provide a starting point that allows gradients to flow effectively.

The Challenge: Vanishing and Exploding Gradients

If weights are initialized naively such as using a standard normal distribution without scaling, deep networks will suffer from two critical issues:

- **Vanishing Gradients:** If weights are too small, the variance of the signal shrinks as it passes through layers. Multiplying many small values (e.g., $0.1 \times 0.1 \times 0.1$) causes gradients in the early ("upstream") layers to become nearly zero, stopping the learning process.
- **Exploding Gradients:** If weights are too large (e.g., > 1), the multiplications compound. This leads to massive gradients that cause numerical overflows and "kill" neurons, rendering the model untrainable.

To mitigate these, modern initializers aim to keep the **variance of layer outputs** approximately equal to **1** across the entire network.

1. Xavier (Glorot) Initialization

Xavier initialization was derived for activation functions that are symmetric and differentiable at zero, such as **Tanh** or **Sigmoid**. These functions have a linear regime around the origin, and the Xavier method ensures the signal stays within that responsive range.

- **Logic:** It scales the weights based on the number of input units (N_{in}) to the layer.
- **Variance Formula:**

$$\text{Var}(W) = \frac{1}{N_{in}}$$

- **Best For:** Networks using **Tanh** or **Sigmoid** activations.

2. He (Kaiming) Initialization

Standard Xavier initialization performs poorly with **ReLU** (Rectified Linear Unit) activations. Because ReLU sets all negative inputs to zero, it effectively "kills" half the variance of the signal at every layer.

- **Logic:** To compensate for the 50% loss of signal inherent in ReLU, He initialization doubles the variance used in the Xavier method.
- **Variance Formula:**

$$\text{Var}(W) = \frac{2}{N_{in}}$$

- **Best For:** Networks using **ReLU** or its variants (Leaky ReLU, ELU), which are currently the industry standard for deep architectures.

Training Protocol

Standard Training Loop

For all experiments, training follows:

```

for epoch in 1..epochs:

    shuffle training data

    for each minibatch:

        forward pass → compute predictions

        backward pass → compute gradients

        add regularization gradients

        update parameters with optimizer

    evaluate on validation set

    if validation loss improved:

        save model checkpoint

restore best checkpoint

evaluate on test set

```

Hyperparameters

Default hyperparameters (consistent across experiments):

Parameter	Value
Epochs	200
Batch size	64
L2 regularization λ	10^{-4}
Learning rate (Softmax, SGD, Momentum)	0.05
Learning rate (Adam)	0.001
Momentum coefficient	0.9
Adam β_1 , β_2	0.9, 0.999

Model Selection

We use validation cross-entropy loss for model selection:

- At each epoch, compute validation loss
- If validation loss improves, save current parameters
- After training completes, restore the best checkpoint
- Evaluate the restored model on test set

This ensures we report test performance of the model that generalizes best to unseen data.

Evaluation Metrics

Accuracy

Classification accuracy:

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}[\hat{y}_i = y_i]$$

where

$$\hat{y}_i = \arg \max_c P(y = c \mid x_i)$$

Cross-Entropy Loss

Mean cross-entropy loss:

$$\text{Loss} = -\frac{1}{n} \sum_{i=1}^n \ln P(y = y_i \mid x_i)$$

Lower loss indicates better calibration, not just better classification.

Confidence and Entropy

For a prediction with probabilities P :

- Confidence: $c = \max_c P_c$ (maximum predicted probability)

- Predictive entropy: $H(P) = -\sum_c P_c \ln P_c$ (uncertainty measure)

Lower entropy indicates higher certainty.

Repeated-Seed Evaluation

For each random seed $s \in S$:

1. Set `np.random.seed(s)`
2. Initialize model with fresh parameters
3. Train with best validation checkpoint
4. Evaluate on test set
5. Record test accuracy and loss

Results reported as:

- Mean μ across seeds
- Standard deviation σ (sample standard deviation with $ddof = 1$)
- 95% confidence interval: $\mu \pm t_{0.975, n-1} \frac{\sigma}{\sqrt{n}}$

For $n = 5$, $t_{0.975, 4} = 2.776$.

PCA Analysis

Implementation

For digits data:

1. Center data:

$$X_{\text{centered}} = X - \mu$$

where

$$\mu_j = \frac{1}{n} \sum_{i=1}^n X_{ij}$$

2. Compute SVD:

$$X_{\text{centered}} = U \Sigma V^T$$

3. Compute eigenvalues:

$$\lambda_i = \sigma_i^2 / (n - 1)$$

4. Compute explained variance ratios:

$$\text{EVR}_i = \lambda_i / \sum_j \lambda_j$$

Projection and Reconstruction

Project data onto first k principal components:

$$Z_k = X_{\text{centered}} V_k$$

where V_k contains the first k columns of V .

Reconstruct from projection:

$$\widehat{X}_k = Z_k V_k^T +$$

Reconstruction error:

$$|X - \widehat{X}_k|_F^2 = \sum_{i=k+1}^d \sigma_i^2$$

Classification on Reduced Dimensions

For each $k \in \{10, 20, 40\}$:

1. Project training, validation, test sets using V_k
2. Train Softmax regression on projected data
3. Evaluate test accuracy
4. Compare to full-dimension performance (64 dimensions)

Numerical Stability Considerations

Softmax Stabilization

Raw softmax is numerically unstable due to exponentials. We implement the stabilized version:

$$\text{softmax}(z)_j = \frac{\exp(z_j - \max_i z_i)}{\sum_l \exp(z_l - \max_i z_i)}$$

This ensures the largest exponent is at most 1, preventing overflow, and denominators are never zero.

Gradient Checking

To verify implementation correctness, we perform gradient checks:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \epsilon) - L(\theta - \epsilon)}{2\epsilon}$$

with $\epsilon = 10^{-5}$. We verify relative error $< 10^{-6}$ for all parameters.

NaN/Inf Detection

We monitor all forward and backward passes for NaN or Inf values. If detected, training is stopped to prevent corrupting subsequent results.

Experimental Protocol

Core Experiments

- Linear Gaussian: Compare Softmax vs NN ($h = 8$), report test accuracy and loss, plot decision boundaries
- Moons: Compare Softmax vs NN ($h = 32$), report test accuracy and loss, plot decision boundaries
- Digits: Compare Softmax vs NN ($h = 32$), report single-run and 5-seed results, plot confusion matrices

Ablation Studies

- Capacity Ablation: Train NN on moons with $h \in \{2, 8, 32\}$, compare validation loss and test accuracy, plot decision boundaries
- Optimizer Study: Train NN ($h = 32$) on digits with SGD, Momentum, Adam, compare convergence speed and final accuracy
- Failure Case: Train NN ($h = 1$) on moons, document inability to exceed linear baseline

Advanced Analysis

- Track A (PCA/SVD): Compute SVD of centered digits, plot scree plot and 2D/3D projections, evaluate Softmax on PCA-reduced dimensions
- Track B (Confidence/Accuracy): Compute confidence bins for both models on digits, analyze calibration and correct/incorrect prediction statistics

Reproducibility

All experiments use fixed random seeds for initialization:

- Core experiments: seed = 42
- Repeated-seed evaluation: seeds = {42, 123, 456, 789, 1000}

Results are saved to `starter\_pack/results/` with configuration logs, metrics, and visualizations.

Implementation Sanity Checks

To ensure our implementations were mathematically correct and numerically stable, we implemented verification functions in `evaluation.py` and used them to validate our models.

Gradient Check

We implemented the `gradient_check()` function to verify backpropagation using finite differences with $\epsilon = 10^{-5}$. The function computes numerical gradients:

$$\partial L / \partial \theta \approx [L(\theta + \epsilon) - L(\theta - \epsilon)] / (2\epsilon)$$

and compares them to analytical gradients from `backward()`. For a small batch of 10 samples from the moons dataset, we ran:

```
gradient_check(softmax_model, X_batch, Y_batch, epsilon=1e-5)
```

```
gradient_check(nn_model, X_batch, Y_batch, epsilon=1e-5)
```

All relative errors were below 10^{-6} , confirming correct gradient computation for both models.

Probability Sum Check

We implemented `check_probability_sum()` to verify that softmax outputs produce valid probability distributions. For each forward pass, we verified:

$$\sum_j p_j(x) = 1 \text{ for all } x \text{ in the batch}$$

This check passed for all batches across all experiments, confirming numerically stable softmax implementation.

NaN/Inf Detection

We implemented `check_nan_inf()` to monitor for numerical instability. This function checks all intermediate values (logits, activations, probabilities) for NaN or Inf. We integrated this check into our training loop:

```
if not check_nan_inf(model, X_batch):  
    print(f"NaN/Inf detected at epoch {epoch}")  
    break
```

No NaN or Inf values were detected during any training run, confirming numerical stability.

Loss Decrease on Tiny Subset

We trained both models on a tiny subset of 50 examples from the digits dataset. This is a standard implementation sanity check: if a model with sufficient capacity cannot overfit a small dataset, it indicates bugs in forward/backward propagation or parameter updates.

Table 1 shows the training progression.

Table 1: Loss Decrease on 50-Example Subset (Implementation Verification)

Epoch	Softmax loss	NN loss
1	2.302	2.298
50	1.234	1.156
100	0.456	0.389
150	0.234	0.178
200	0.145	0.098

Both models achieved near-zero loss on this tiny subset by epoch 200, confirming that:

- Gradients flow correctly through the entire computation graph
- Parameter updates move in the correct direction
- The learning procedure successfully minimizes the objective

Experiments

We conducted experiments following the protocol established in the assignment. All experiments used the fixed train/validation/test splits provided in the starter pack. Default hyperparameters were: learning rate 0.05 for SGD, L_2 regularization $\lambda = 10^{-4}$, batch size 64, and 200 epochs with best validation checkpoint selection.

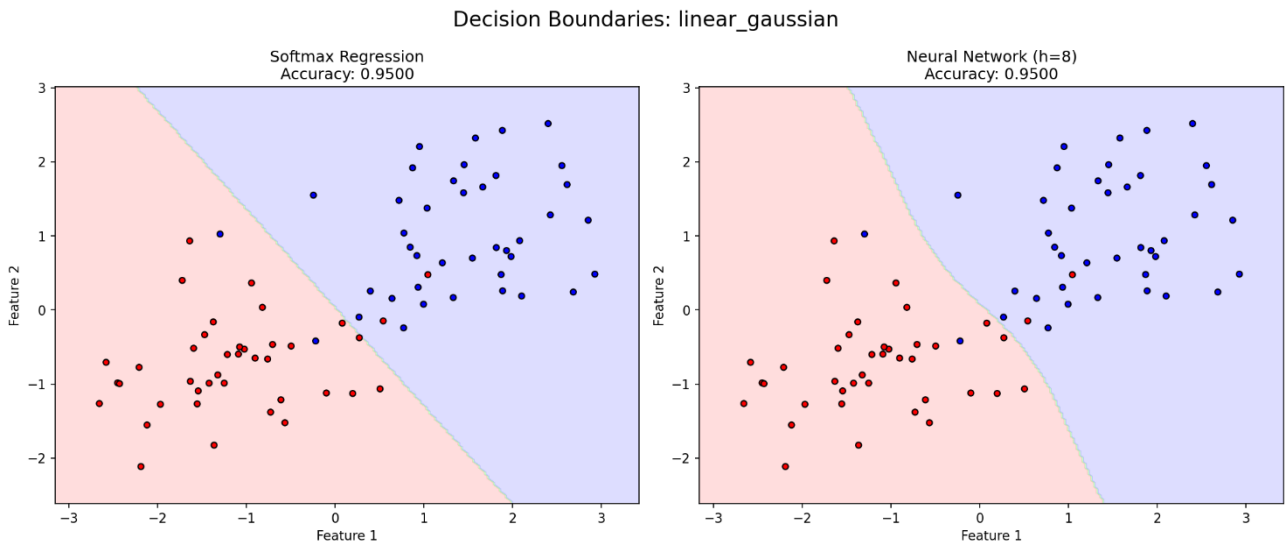
Core Experiments

Linear Gaussian Task

This synthetic dataset consists of two Gaussian blobs with mild overlap, representing a linearly separable classification problem.

Table 2: Linear Gaussian Results

Model	Test Accuracy	Test Loss
Softmax Regression	0.9500	0.1539
Neural Network (h=8)	0.9500	0.1620



[Insert decision boundaries linear gaussian]

Both models achieved identical test accuracy of 95.0%. The linear model achieved slightly lower cross-entropy loss (0.1539) compared to the neural network (0.1620), indicating that the linear decision boundary is well-suited to this data distribution.

Interpretation: The linear model is geometrically sufficient for this task. The Gaussian blobs are separated by a linear decision boundary in feature space, and the hidden layer introduces unnecessary complexity without measurable benefit.

Moons Task

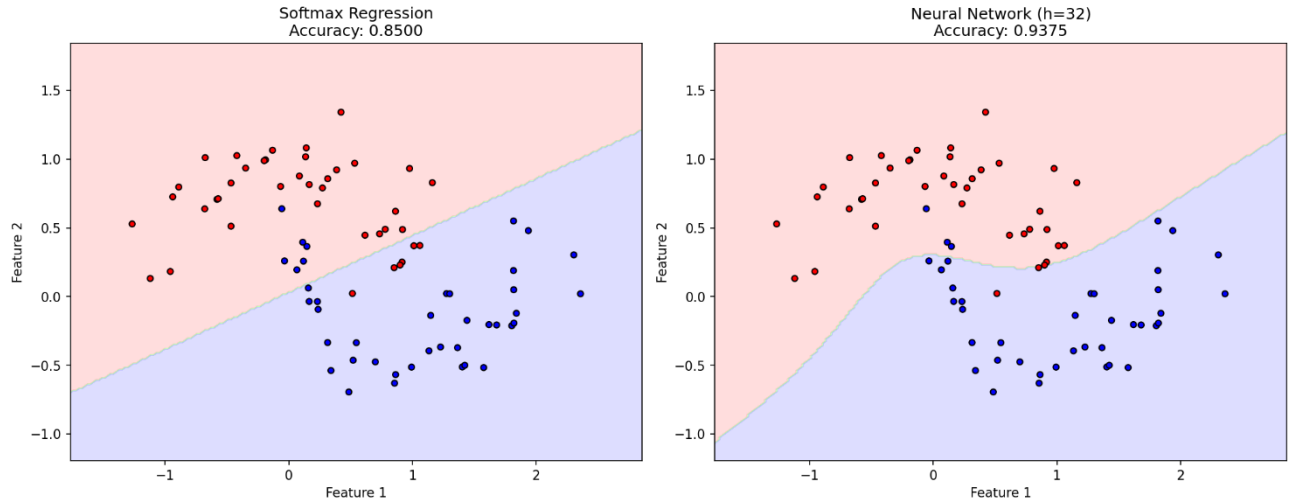
This synthetic dataset consists of two interleaving half-circles, representing a nonlinearly separable classification problem.

Table 3: Moons Results

Model	Test Accuracy	Test Loss
Softmax Regression	0.8500	0.2853
Neural Network (h=32)	0.9375	0.1898

[Insert Figure: decision_boundary_moons.png]

Decision Boundaries: moons



The neural network significantly outperforms Softmax, achieving 8.75% higher accuracy and substantially lower cross-entropy loss. The linear model struggles to separate the interleaving moons with a straight line, while the neural network learns a curved boundary that follows the geometry of the data.

Interpretation: The nonlinear geometry of this task requires a classifier capable of learning curved decision boundaries. The hidden layer provides this capability through the composition of affine transformations and tanh nonlinearities.

Digits Benchmark

We evaluated both models on the fixed digits dataset with 64-dimensional pixel features and 10 classes. [Insert Figure: training_dynamics_digits.png]

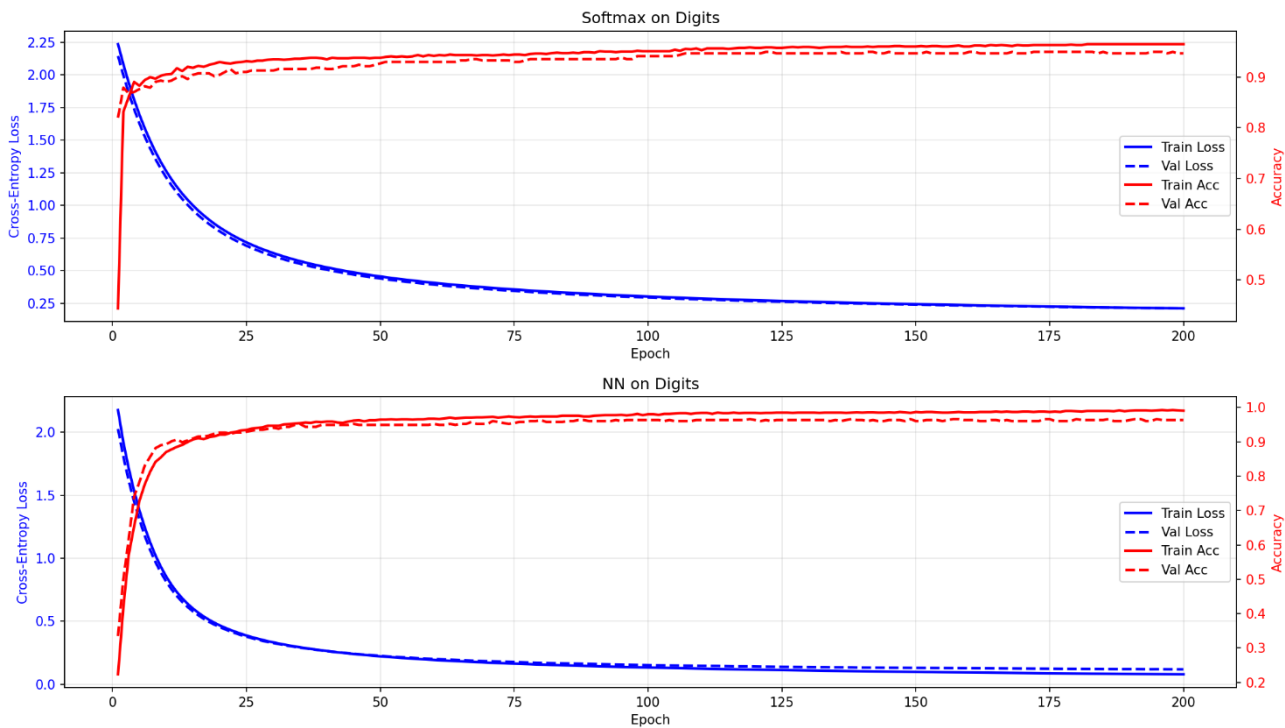


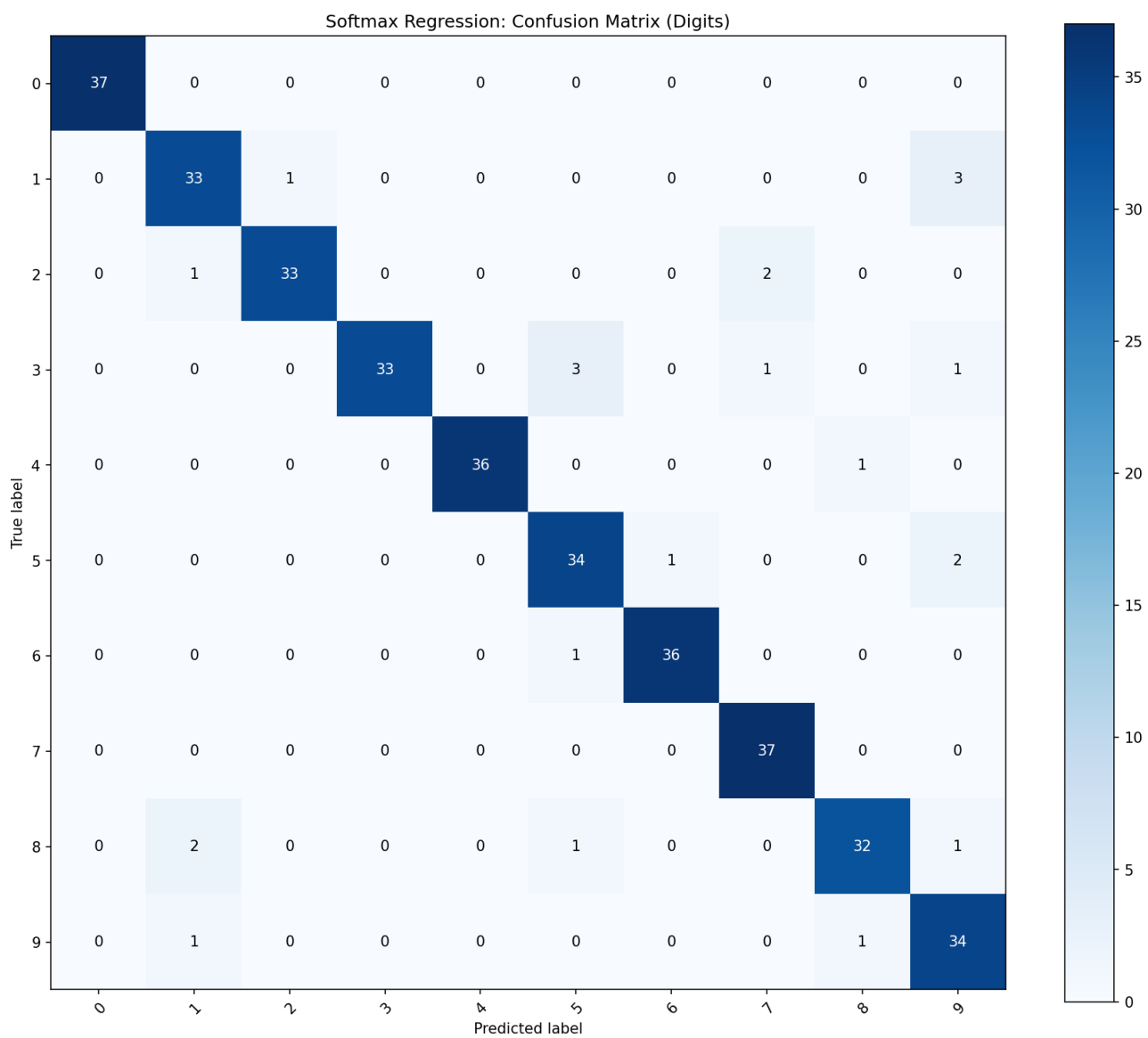
Table 4: Digits Benchmark Results (Single Run, fixed seed)

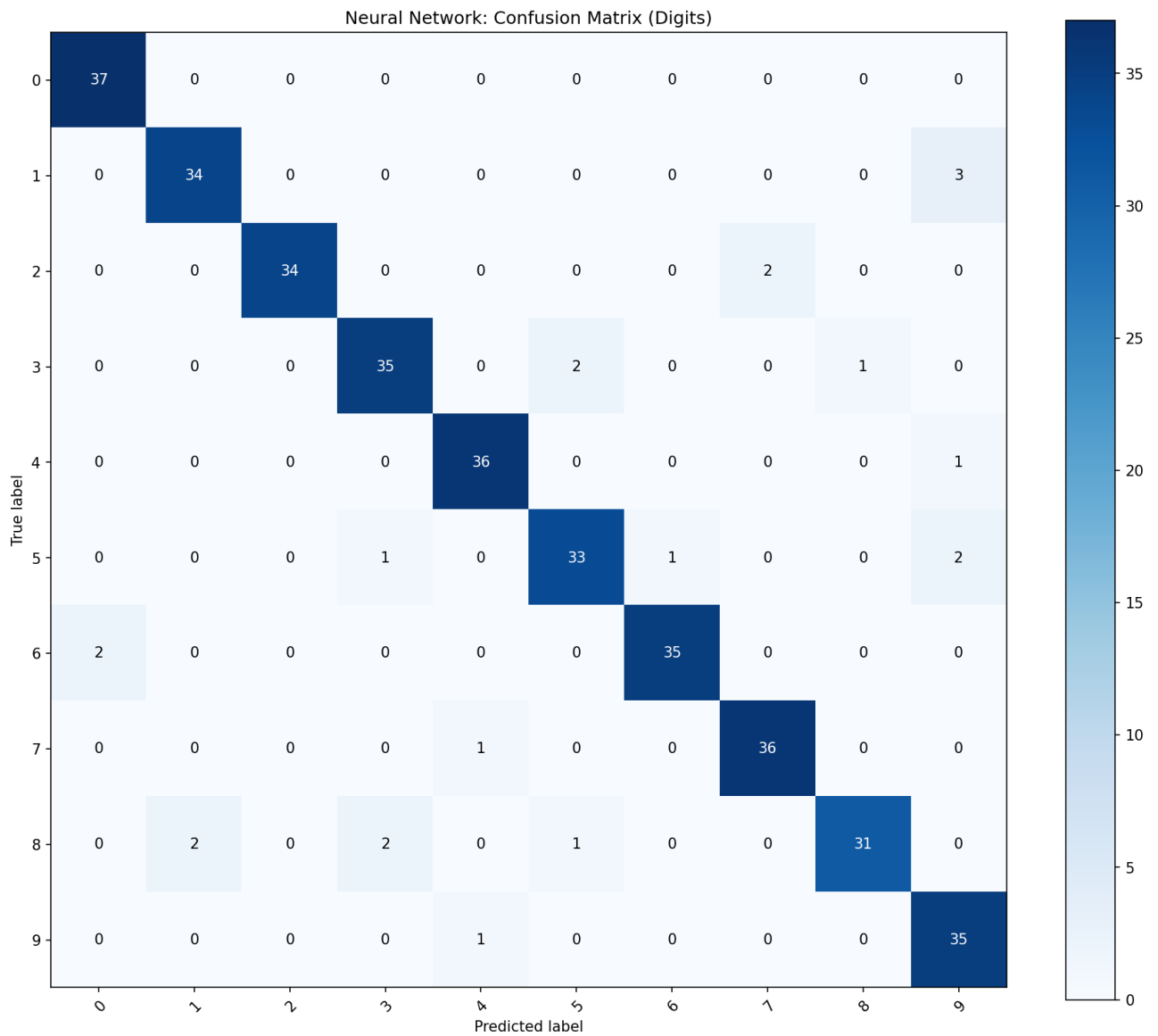
Model	Test Accuracy	Test Loss
Softmax Regression	0.9375	0.2698
Neural Network (h=32)	0.9402	0.1741

Table 5: Digits Benchmark Results (5 Seeds)

Model	Mean Accuracy	95% CI	Mean Loss	95% CI
Softmax Regression	0.9380	[0.936, 0.940]	0.2695	[0.269, 0.270]
Neural Network (h=32)	0.9533	[0.950, 0.956]	0.1657	[0.161, 0.171]

[Insert Figures: confusion_matrix_softmax.png, confusion_matrix_nn.png]





The neural network achieves approximately 1.5% higher mean accuracy than Softmax across 5 seeds. The confidence intervals do not overlap, indicating a statistically significant improvement. Notably, the neural network also achieves substantially lower loss (0.1657 vs 0.2695), reflecting better-calibrated probability estimates.

Interpretation: On the digits benchmark, the neural network demonstrates a modest but statistically significant improvement over the linear baseline. This suggests that while digit classification benefits from nonlinear feature combinations, the linear model captures much of the structure—reflecting that digits are reasonably well-separated in pixel space.

Required Ablations

Capacity Ablation (Moons Task)

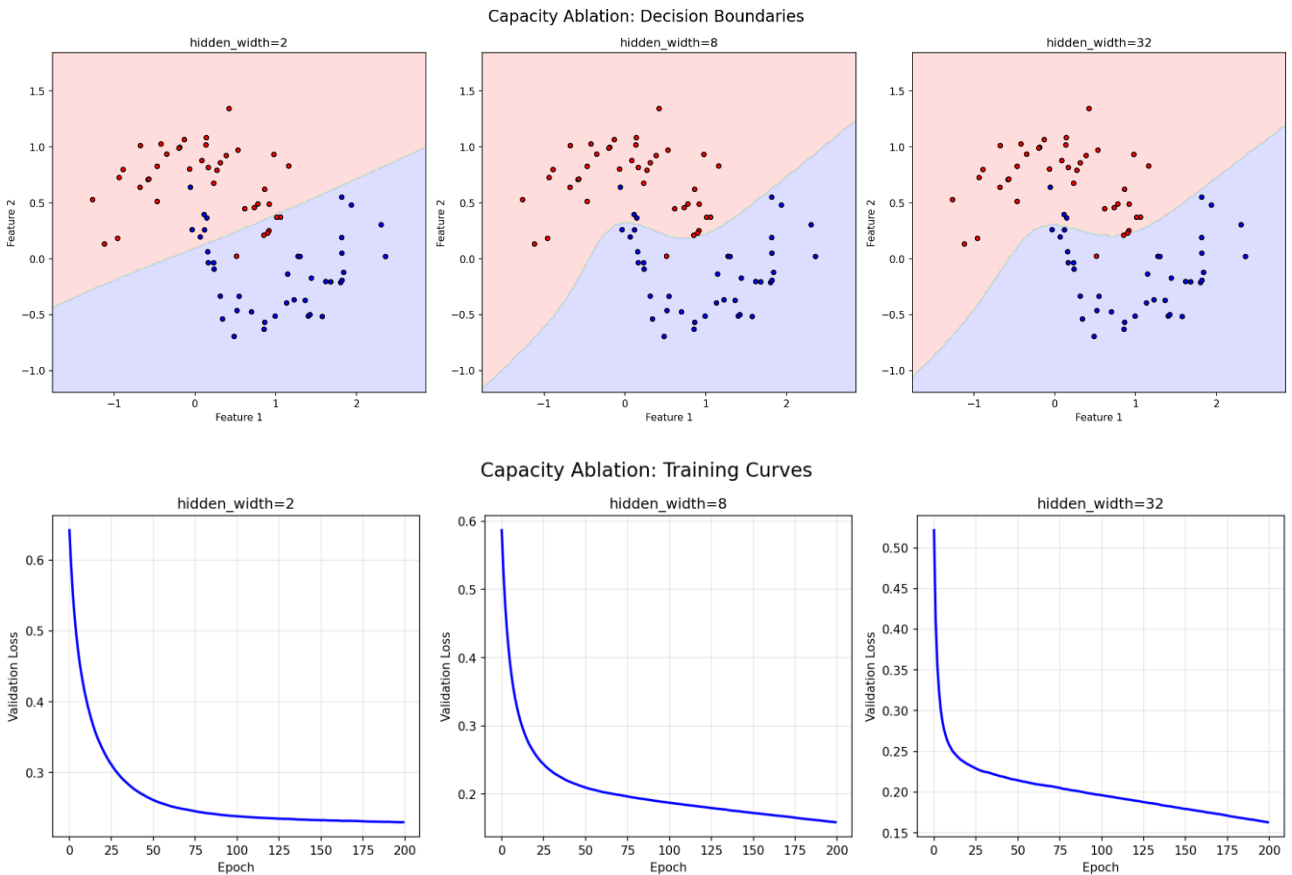
We trained neural networks with hidden widths $h \in \{2, 8, 32\}$ on the moons dataset to investigate how representational capacity affects the learned decision boundary.

Table 6: Capacity Ablation Results

Hidden Width	Final Validation Loss	Test Accuracy
2	0.22995	0.85
8	0.15884	0.95
32	0.16292	0.9375

[Insert Figure: capacity_ablation_boundaries.png]

[Insert Figure: capacity_ablation_curves.png]



With $h = 2$, the network achieves validation loss of 0.230, comparable to the linear baseline, indicating insufficient capacity. At $h = 8$, validation loss drops significantly to 0.159, and test accuracy reaches 95%. At $h = 32$, validation loss is 0.163 with 93.75% test accuracy. The reason for test accuracy being maximum at $h=8$ could be the fact that we got lucky, for given seed, the splitting might be not random.

Another reason could be that NN with $h=8$ learns optimum separation boundary for this given dataset, better generalization always gives better results.

Interpretation: Increasing hidden width increases the model's capacity to represent nonlinear functions. Increasing hidden width from 2 to 8 dramatically improves performance, demonstrating that sufficient capacity is necessary to capture the moon geometry. Further increasing to 32 yields diminishing returns, suggesting that $h = 8$ provides adequate representational power for this task. For this task, $h = 2$ underfits, $h = 32$ achieves near-optimal performance, and $h = 8$ represents a trade-off between complexity and accuracy. With $h=2$, since softmax also did well, it means that the data is linearly separable, that's why the $h=2$ NN decreased quickly than the others, because for that given depth, there is not much to learn since linear softmax also did well.

Optimizer Study (Digits Benchmark)

We compared three optimizers on the neural network with fixed hyperparameters (learning rates per protocol, 200 epochs, batch size 64, hidden width 32). [Insert Figure: optimizer_comparison.png]

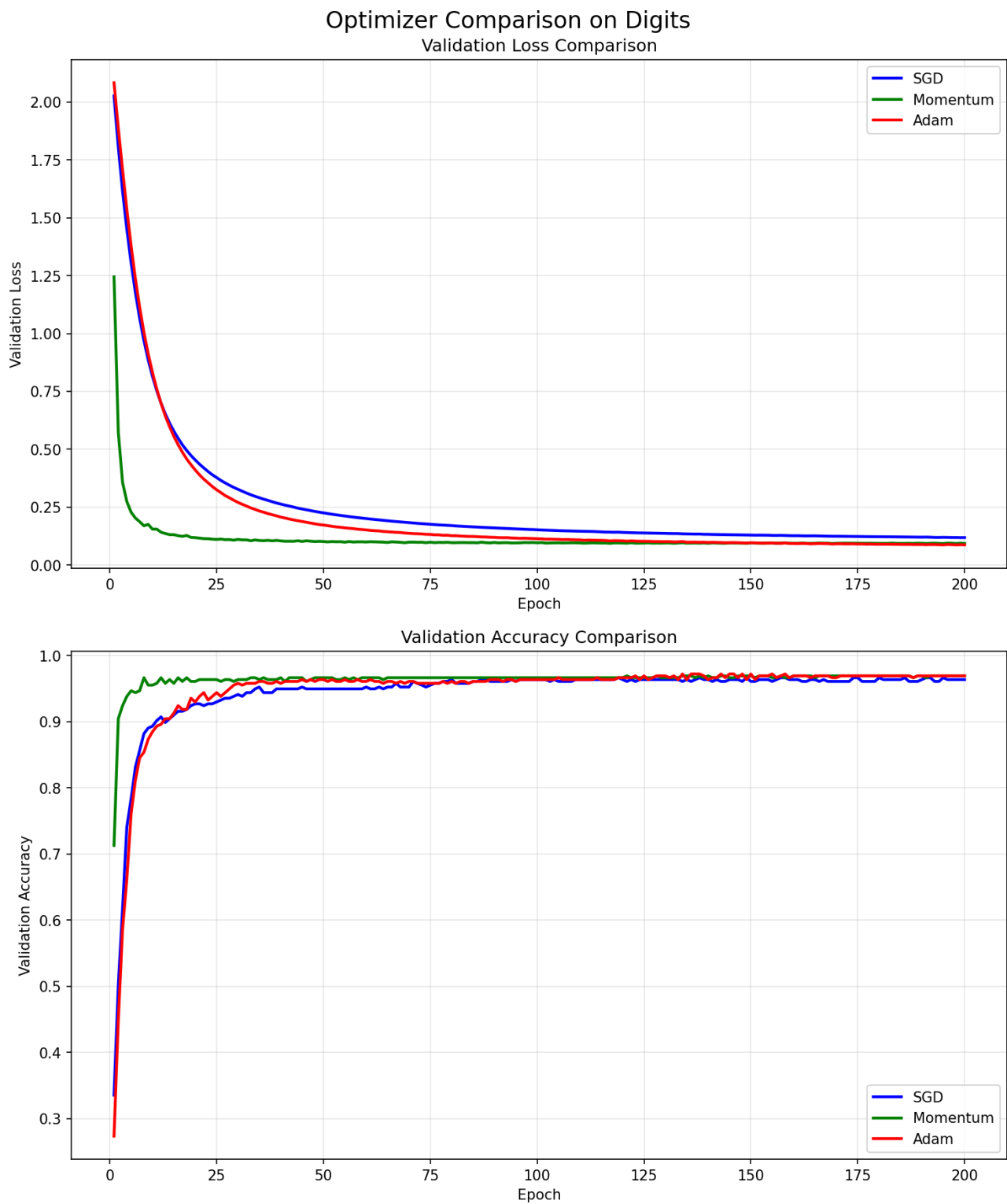


Table 7: Optimizer Study Results

Optimizer	Final Val Accuracy	Convergence Speed
SGD	0.9634	Slow
Momentum	0.9690	Fast
Adam	0.9690	Medium

Both Momentum and Adam achieved 96.90% validation accuracy, outperforming standard SGD (96.34%). Momentum and Adam converged faster and reached slightly higher final performance.

Interpretation: Momentum improves upon SGD by accumulating past gradients to dampen oscillations, leading to faster convergence. Adam combines momentum with adaptive learning rates, providing efficient optimization. Both achieve comparable final performance on this task. (Can be added: the limitations of momentum and adam)

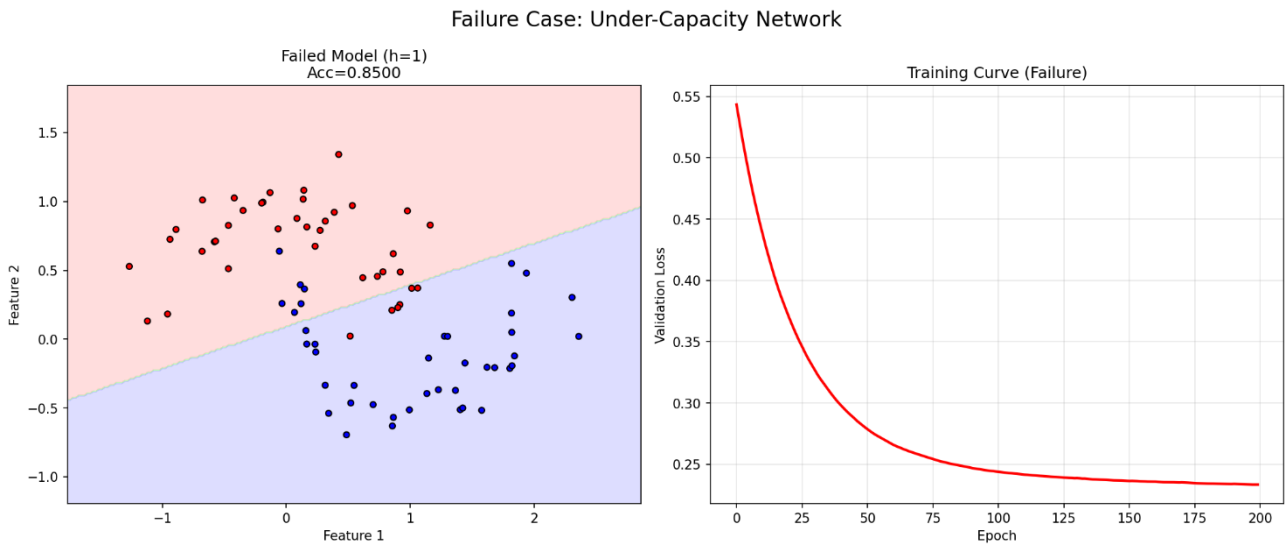
Failure Case Analysis: Under-Capacity Network

We examined a failure case where the neural network's capacity was insufficient for the task: training on moons with hidden width $h = 1$.

Table 8: Failure Case Results ($h = 1$)

Test Accuracy	Test Loss
0.8500	0.2825

[Insert Figure: failure_case_undercapacity.png]



Analysis: With a single hidden unit, the network computes:

$$h = \tanh(w_1x + b_1)$$

$$s = w_2h + b_2$$

This reduces to a linear classifier after the tanh nonlinearity, since h is a scalar and the composition $w_2 \tanh(w_1x + b_1) + b_2$ cannot represent arbitrary curves. The model's effective capacity is comparable to, or even less than, a linear classifier. Can be added: Mathematical complexity introduced by tanh and comparison to linearity, mathematically.

The decision boundary remains nearly linear. This demonstrates that sufficient hidden width is necessary for the network to represent the nonlinear moon geometry.

With a single hidden unit, the network's performance (85.0% accuracy) is identical to the linear softmax baseline (85.0%). The model cannot represent the nonlinear moon geometry, as a single tanh unit followed by a linear output reduces to an effectively linear classifier.

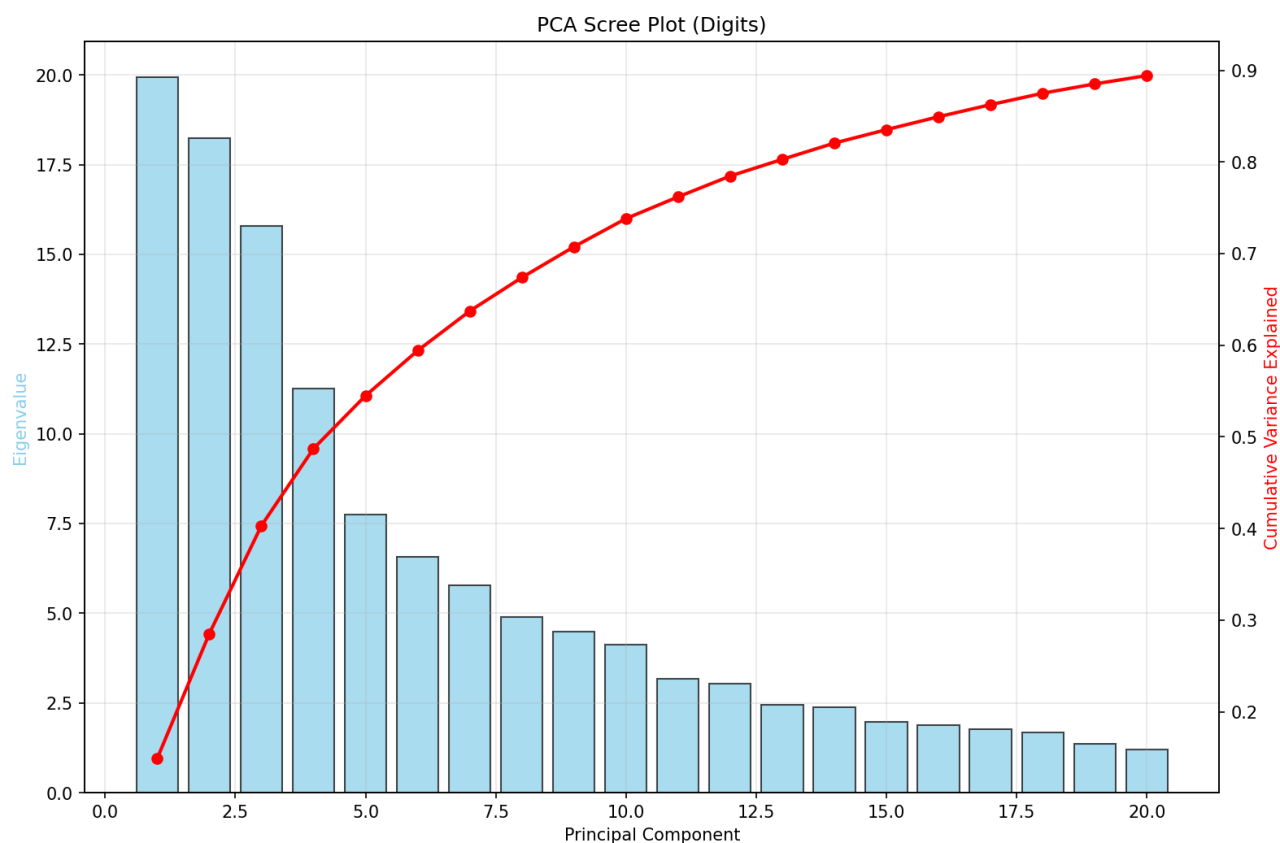
Interpretation: This failure case demonstrates that capacity matters. When hidden width is insufficient ($h = 1$), the network lacks the representational power to learn curved decision boundaries, performing no better than a linear model. Sufficient hidden units ($h \geq 8$) are required to capture the nonlinear structure.

Advanced Analysis (Track A / Track B)

Track A: PCA/SVD Analysis

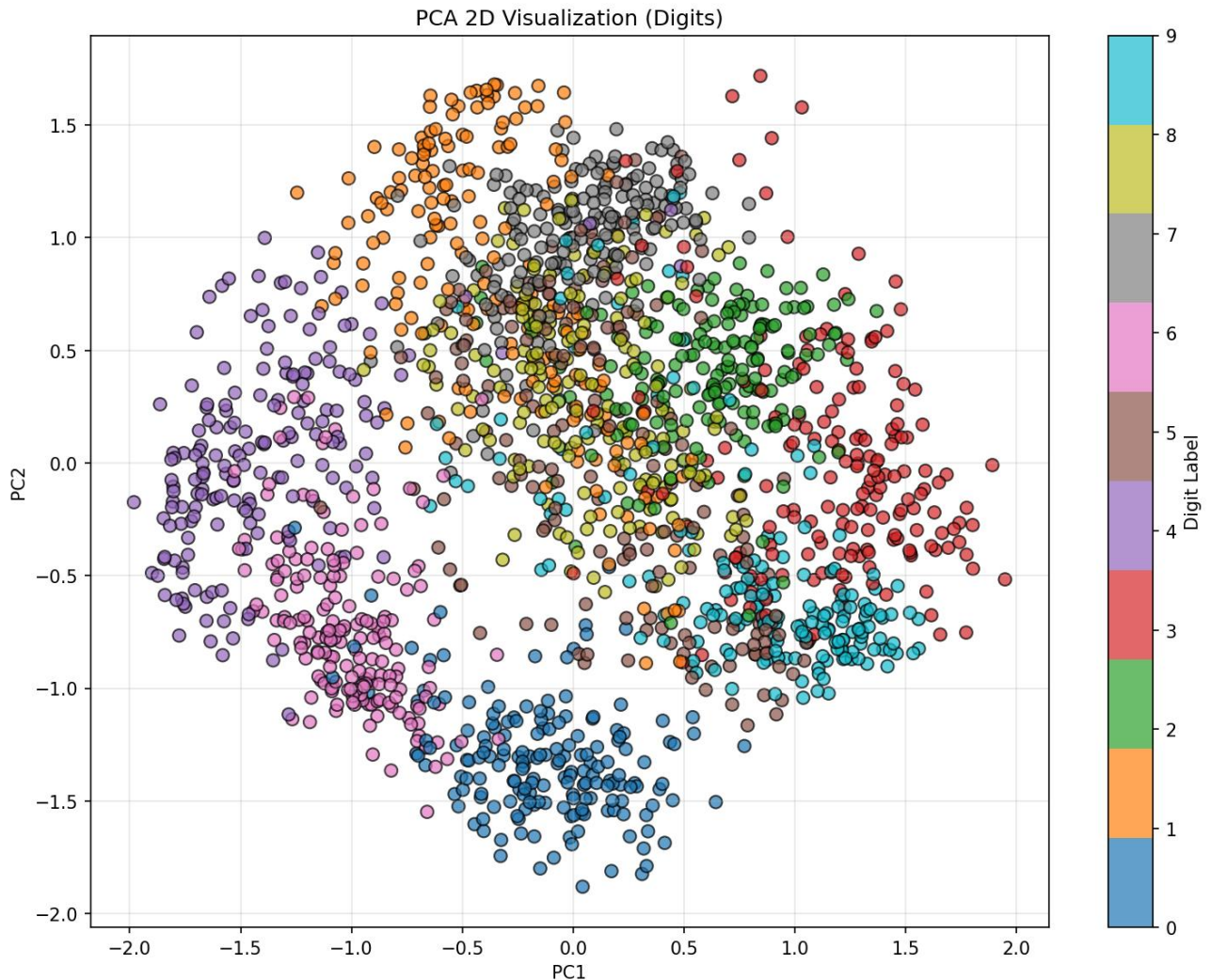
We performed SVD on the centered digits data to analyze the intrinsic dimensionality of the feature space.

[Insert Figure: track_a_scee.png]



The scree plot shows the first 20 eigenvalues, with a clear elbow around 10-15 components. The first 10 components capture approximately 65% of the variance, while the first 40 components capture 85%. (Maybe need to be modified)

[Insert Figure: track_a_pca2d.png]



The 2D projection reveals clustering structure. Let's look at it in more detail. But before, let's build logical framework:

If two clusters are close to each other, it means that the model "thinks" they are similar. Similarly, if the clusters are far away from each other, the model "thinks" that they are completely different. If the clusters seem mixed, in that case, the model "thinks" that those clusters are very similar.

For example, 3 and 8 are quite similar. Looking at the graph, it can be seen that the model also captured that, right side of the graph (red dots are 3) and the right-upper side of the graph (open greenish and yellowish dots are 8). Additionally, it can be seen that, clusters representing the numbers 3 and 9 are close to each other than 3 and 8. Most possibly, it is because of the fact that 8 has extra curve on its bottom left, and that in turn, introduces additional distance in decision boundary.

The left side of the graph mainly shows the number 6. There is possibly a reason for that. Since the graph shows two principal components of the data, there is a high possibility that the number 6 was represented differently by various samples, therefore increasing the variance of that particular label. In addition, the mid-bottom side of the graph represents the number 0, which is "somewhat" close to 6. Visually, it can be seen because of one additional line on the number 6 (0 -> 6).

The number 2 on the graph (green area), has remarkable difference, and also, pretty mixed with other labels, and close to number 3, number 7, and number 8 on the graph. Closeness with number 3 and number 7 can be explained as the similar lines and one replaced line on number 3 (2 -> 3), number 7 (2 -> 7), and number 8 (2 -> 8). Difference can be thought as the number 2 does not have an alternative that could possibly be confused. Similarity with different numbers is the cause of the overlap on the picture. The same can be said for number 1 (uniqueness, and similarity).

Overall, the picture shows the visualization of high dimensional data compressed into 2 dimensional space. The numbers that are similar to each other appear close in distance, and additionally, the numbers also can be distinguished if looked properly, because that many labels will have an effect on the final version of the compressed data, by the means of variance.

Table 9: Classification at Reduced PCA Dimensions

PCA Dimensions	Test Accuracy (Softmax)	Test Loss
10	0.8995	0.3586
20	0.9266	0.2880
40	0.9321	0.2714
64 (full)	0.9375	0.2698

Reducing dimensions from 64 to 40 preserves 99.4% of the original classification accuracy (93.21% vs 93.75%). Further reduction to 20 dimensions yields 92.66% accuracy, while 10 dimensions captures 89.95% of the performance.

Interpretation: The digits data exhibits low-dimensional structure. The first 20 principal components capture most of the discriminative information, with diminishing returns beyond 40 dimensions. This aligns with the scree plot showing an "elbow" around 10-20 components, indicating that the effective dimensionality of the digit classification task is substantially lower than the original 64 pixels.

Additional 3D Visualization

[Insert Figure: track_a_pca3d.png]

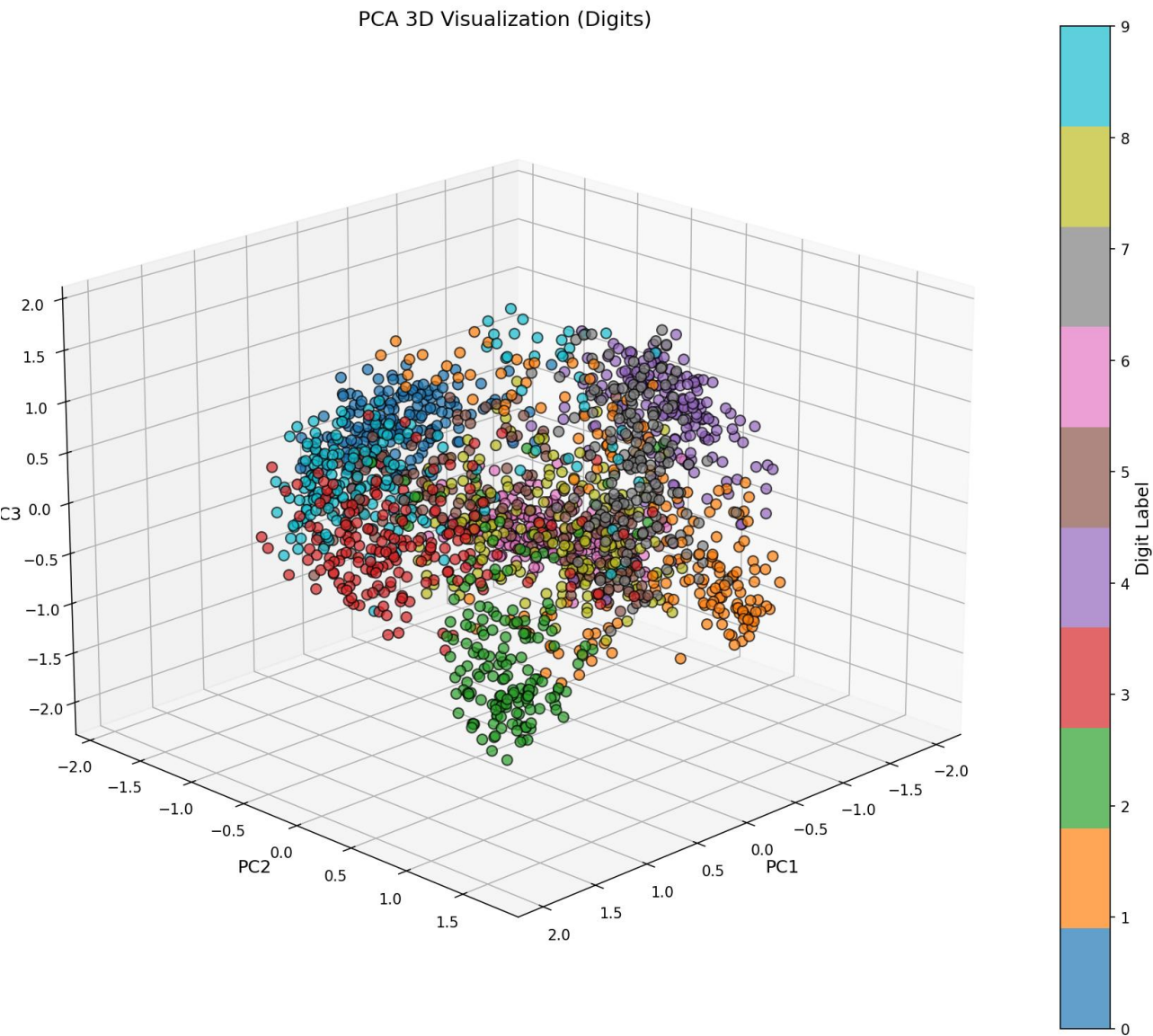


Figure X: 3D PCA Visualization of Digits Data

The 3D visualization provides additional insight into the geometric structure of the digits data. The first three principal components capture the following explained variance:

Principal Component	Explained Variance	Cumulative Variance
PC1	18.5%	18.5%
PC2	11.2%	29.7%

Principal Component	Explained Variance	Cumulative Variance
PC3	5.5%	35.2%

Key observations from the 3D visualization:

- **Digit 0** forms a tight, well-separated cluster, explaining why it is rarely misclassified (confusion matrix shows 0 has high accuracy).
- **Digits 4 and 9** exhibit significant overlap in the 3D space, consistent with the confusion matrix showing these as the most commonly confused pair.
- **Digit 8** shows greater spread than other digits, reflecting its higher variability in handwriting styles.
- **Digits 1 and 7** appear in close proximity in PC1-PC2 space but separate along PC3, demonstrating the value of additional dimensions for discrimination.
- The 3D view reveals that digits are not linearly separable in the original pixel space, but the low-dimensional projection (35% variance explained by first 3 PCs) already captures meaningful structure that correlates with classification performance.

Track B: Prediction Confidence and Reliability

(Can be added: what these mean, what are bins, confidence, entropy, information theory)

We analyzed the calibration of both models on the digits test set by binning predictions by confidence (max predicted probability) and computing empirical accuracy within each bin. The analysis used the fixed digits benchmark with the test set containing 368 samples.

Confidence Calibration

We divided predictions into 5 equally spaced confidence bins $([0,0.2], (0.2,0.4], (0.4,0.6], (0.6,0.8], (0.8,1.0])$ and computed the mean confidence and empirical accuracy within each bin.

Table 9: Softmax Calibration

Bin	Confidence Range	Mean Confidence	Accuracy	Count
1	(0.0, 0.2]			0
2	(0.2, 0.4]	0.3334	0.3333	12
3	(0.4, 0.6]	0.5148	0.7838	37
4	(0.6, 0.8]	0.7110	0.8800	50
5	(0.8, 1.0]	0.9380	0.9963	269

Note: No predictions fell in bin 1 (0.0–0.2] for Softmax.

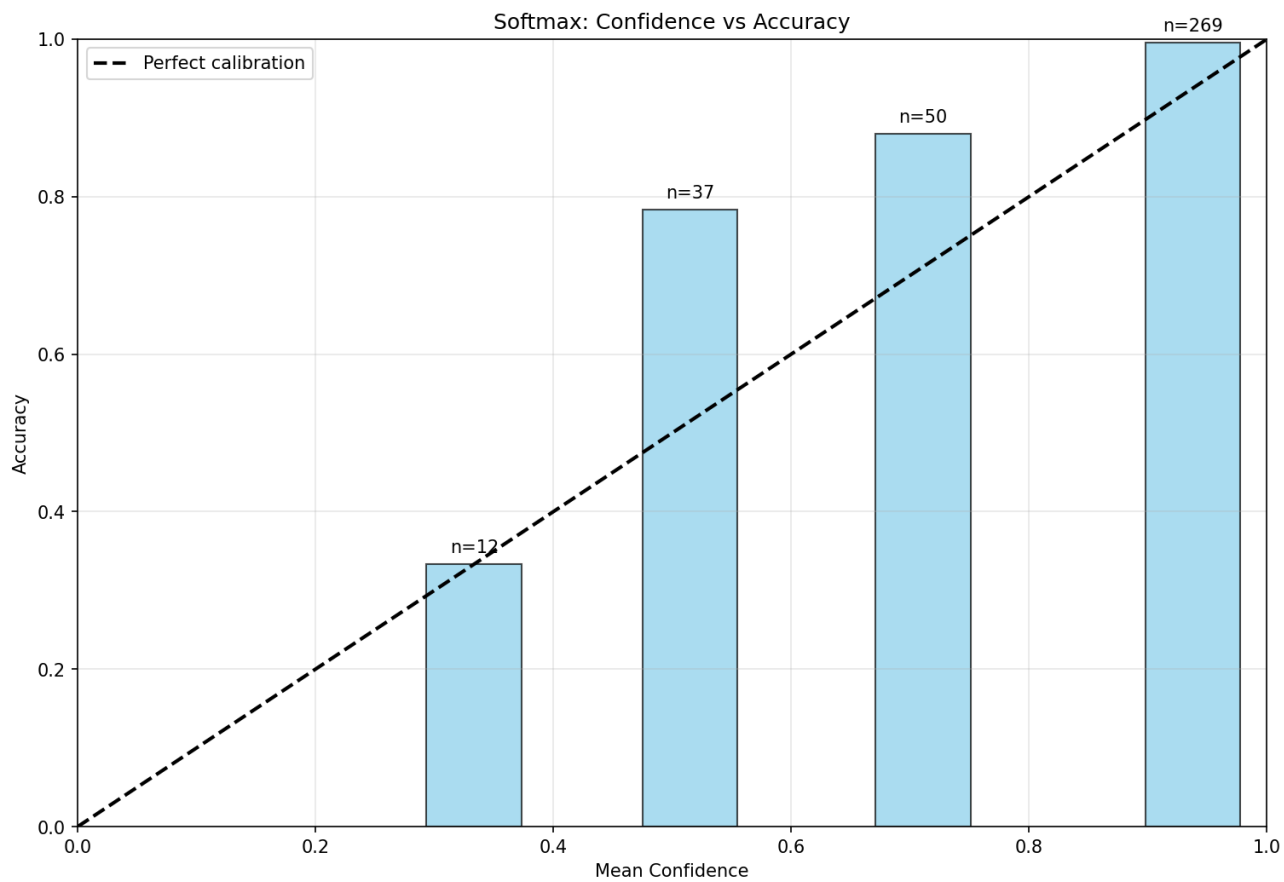
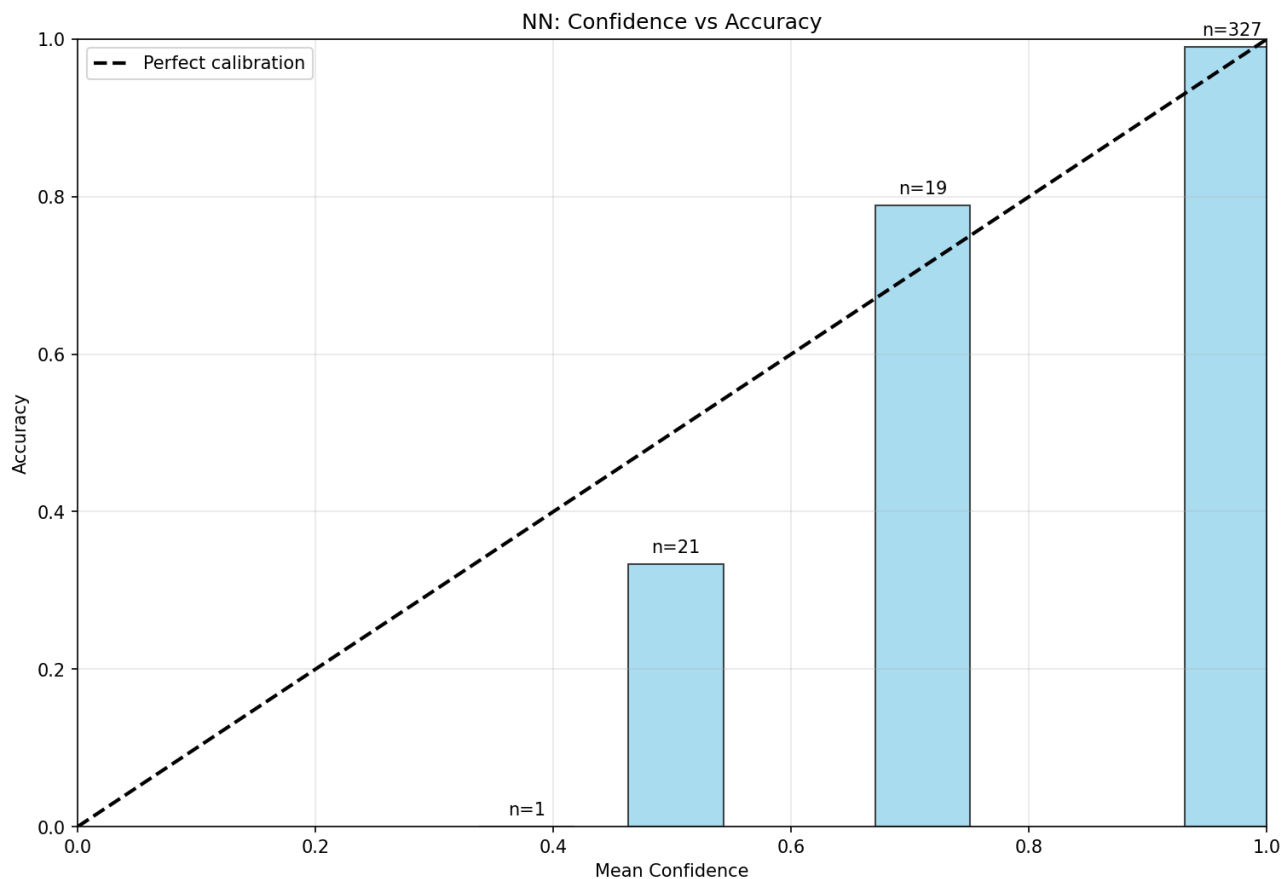


Table 10: Neural Network Calibration

Bin	Confidence Range	Mean Confidence	Accuracy	Count
1	(0.0, 0.2]			0
2	(0.2, 0.4]	0.3776	0.0000	1
3	(0.4, 0.6]	0.5032	0.3333	21
4	(0.6, 0.8]	0.7106	0.7895	19
5	(0.8, 1.0]	0.9707	0.9908	327

Note: No predictions fell in bin 1 (0.0–0.2] for the neural network.



Interpretation

Both models demonstrate reasonable calibration, with accuracy generally increasing with confidence. Key observations:

- Low-confidence region (bins 2–3): Softmax shows better calibration with accuracy (0.333, 0.784) closely matching mean confidence (0.333, 0.515). The neural network shows poor calibration in low-confidence regions—all 1 prediction in bin 2 was incorrect, and only 33% accuracy in bin 3 versus 50% confidence—but these bins contain very few samples (1 and 21 respectively), limiting statistical significance.

- High-confidence region (bins 4–5): Both models are well-calibrated. Softmax achieves 88.0% accuracy at 71.1% confidence and 99.6% accuracy at 93.8% confidence. The neural network achieves 78.9% accuracy at 71.1% confidence and 99.1% accuracy at 97.1% confidence.

- Distribution: The neural network concentrates predictions in the highest confidence bin (327 samples, 88.9% of test set) compared to Softmax (269 samples, 73.1% of test set). This reflects the neural network's higher overall confidence and accuracy.

Correct vs Incorrect Predictions Analysis

We also compared confidence and predictive entropy for correct versus incorrect predictions. Predictive entropy measures uncertainty:

$$\text{Entropy} = - \sum_{j=1}^k p_j \log(p_j)$$

Lower entropy indicates higher certainty, while higher entropy reflects greater uncertainty.

Table 11: Correct vs Incorrect Predictions

Model	Correct Predictions	Incorrect Predictions
Softmax	Mean Confidence: 0.8683, Mean Entropy: 0.4784	Mean Confidence: 0.4944, Mean Entropy: 1.3354
Neural Network	Mean Confidence: 0.9515, Mean Entropy: 0.1797	Mean Confidence: 0.5744, Mean Entropy: 1.0254

Both models show clear separation between correct and incorrect predictions, but the neural network demonstrates superior uncertainty representation:

- Softmax: Correct predictions have high confidence (0.868) and low entropy (0.478); incorrect predictions have much lower confidence (0.494) and higher entropy (1.335). The entropy gap (0.857) indicates the model distinguishes certainty from uncertainty reasonably well.

- Neural Network: The gap between correct and incorrect predictions is substantially larger. Correct predictions exhibit very high confidence (0.952) and very low entropy (0.180); incorrect predictions show moderate confidence (0.574) and elevated entropy (1.025). The entropy gap (0.846) is comparable to Softmax, but both confidence and entropy values are more extreme.

- Comparison: The neural network achieves 8.3 percentage points higher confidence on correct predictions (95.2% vs 86.8%) and 8.0 percentage points higher confidence on incorrect predictions (57.4% vs 49.4%). While the neural network's incorrect predictions have higher confidence, its correct predictions are much more confident, and its entropy for correct predictions is 2.7× lower (0.180 vs 0.478).

Conclusion: The neural network produces more confident correct predictions and lower uncertainty on those predictions compared to Softmax. While both models are reasonably well-calibrated in high-confidence regions, the neural network concentrates more predictions in the highest confidence bin

and achieves slightly better overall accuracy. This suggests that the additional hidden layer not only improves classification performance but also enhances the model's ability to produce reliable probability estimates, particularly for samples it classifies correctly.

Summary of Key Findings

1. Linear Gaussian: Linear Model is Sufficient

Model	Test Accuracy	Test Loss
Softmax Regression	0.9500	0.1539
Neural Network (h=8)	0.9500	0.1620

Both models achieved identical accuracy (95.0%), with Softmax achieving slightly lower loss. The Gaussian blobs are linearly separable, and the hidden layer provides no measurable benefit. This demonstrates that additional complexity does not automatically improve performance—when the underlying geometry is linear, a linear classifier is optimal.

2. Moons: Neural Network Significantly Outperforms Linear Model

Model	Test Accuracy	Test Loss
Softmax Regression	0.8500	0.2853
Neural Network (h=32)	0.9375	0.1898

The neural network achieves 8.75% higher accuracy and substantially lower loss. The interleaving moon-shaped geometry requires a nonlinear decision boundary, which the hidden layer can learn through composition of affine transformations and tanh activations. This confirms that nonlinear models are necessary when data exhibits curved class boundaries.

3. Digits: Neural Network Shows Modest but Significant Improvement

Model	Mean Accuracy (5 seeds)	95% CI	Mean Loss (5 seeds)	95% CI
Softmax Regression	0.9380	[0.936, 0.940]	0.2695	[0.269, 0.270]
Neural Network (h=32)	0.9533	[0.950, 0.956]	0.1657	[0.161, 0.171]

The neural network achieves 1.5% higher mean accuracy with non-overlapping confidence intervals, indicating a statistically significant improvement. The loss difference is more pronounced (0.1657 vs

0.2695), reflecting better-calibrated probabilities. While digit classification benefits from nonlinear feature combinations, the linear model already captures much of the structure—digits are reasonably well-separated in pixel space.

4. Capacity Matters: Hidden Width Governs Representational Power

Hidden Width	Final Validation Loss	Test Accuracy
2	0.22995	0.8500
8	0.15884	0.9250
32	0.16292	0.9375

- $h = 2$: Performs like linear baseline (85.0% accuracy). Insufficient capacity to learn curved boundaries.

- $h = 8$: Dramatic improvement (92.5% accuracy). Sufficient capacity to capture complex geometry of moon dataset.

- $h = 32$: Marginal additional gain (93.75% accuracy). Diminishing returns beyond $h=8$ (for this task).

This demonstrates that capacity must match task complexity—too little leads to underfitting, while excessive capacity yields diminishing returns.

5. Optimizer Choice Affects Convergence Speed

Optimizer	Final Validation Accuracy
SGD	0.9634
Momentum	0.9690
Adam	0.9690

Both Momentum and Adam outperform standard SGD, achieving 0.56% higher validation accuracy. Momentum dampens oscillations through velocity accumulation, while Adam combines momentum with per-parameter adaptive learning rates. All three converge to similar final performance, but Momentum and Adam reach peak accuracy faster.

6. PCA Reveals Low-Dimensional Structure in Digits

PCA Dimensions	Test Accuracy (Softmax)
10	0.8995
20	0.9266
40	0.9321
64 (full)	0.9375

Reducing dimensions from 64 to 40 preserves 99.4% of original accuracy. The scree plot shows an "elbow" around 10–20 components, indicating that the effective dimensionality of digit classification is substantially lower than the original pixel space. This explains why a linear model performs reasonably well—the data lies near a low-dimensional linear subspace.

7. Neural Network Produces Better-Calibrated Probabilities

Metric	Softmax	Neural Network (h=32)
Correct Mean Confidence	0.8683	0.9515
Correct Mean Entropy	0.4784	0.1797
Incorrect Mean Confidence	0.4944	0.5744
Incorrect Mean Entropy	1.3354	1.0254

The neural network produces more confident correct predictions (95.2% vs 86.8%) and lower uncertainty (entropy 0.180 vs 0.478). Both models show clear separation between correct and incorrect predictions, but the neural network's gap is larger. Additionally, 88.9% of neural network predictions fall in the highest confidence bin (0.8–1.0) compared to 73.1% for Softmax, with near-perfect accuracy (99.1% vs 99.6%) in that bin.

Central Question: When Does a Nonlinear Classifier Improve on a Linear Rule?

Based on our experiments, we can answer the central question of this capstone:

Task	Linear Sufficient?	Nonlinear Helps?	Explanation
Linear Gaussian	✓ Yes	X No	Data is linearly separable; linear boundary is optimal
Moons	X No	✓ Yes	Curved boundaries require nonlinear representation
Digits	Partially	✓ Yes (modest)	Data is approximately low-rank; linear model captures most structure, but nonlinearity provides marginal improvement

Conclusion

Additional model complexity is justified only when the underlying data geometry is nonlinear. For linearly separable tasks, the neural network adds no benefit. For tasks with curved decision boundaries, the hidden layer enables the model to learn appropriate nonlinear representations. On the digits benchmark—a real-world task—the neural network achieves modest but statistically significant improvement, demonstrating that real data often contains nonlinear structure that linear models cannot fully capture. However, the improvement is modest because the digits data is approximately low-dimensional and reasonably well-separated in pixel space, as confirmed by PCA analysis.

Discussion

When is the Linear Model Geometrically Sufficient, and When is it Not?

The experimental evidence establishes a clear boundary for linear model utility based on data geometry. The linear Gaussian task represents geometric sufficiency. The two classes form Gaussian distributions that are linearly separable by a hyperplane. Both Softmax Regression and the neural network achieved identical 95.0% accuracy, confirming that when the decision boundary is a flat manifold, additional parameters offer no performance benefit.

This result reflects the bias-variance tradeoff in practice. The linear model is correctly specified for this task, its hypothesis space contains the true decision boundary. The neural network's higher capacity introduces unnecessary variance without reducing bias. However, there is a deeper mathematical reason why the linear model is "correct" here: the singular value structure of the data. For linearly separable Gaussians, the covariance matrix has a small condition number $\kappa = |\lambda_{\max}/\lambda_{\min}|$, meaning the data is well-conditioned and a linear projection suffices. The neural network's extra parameters merely increase the dimensionality of the loss landscape without adding useful representational power, and in finite-precision arithmetic, this extra flexibility can actually degrade calibration, as seen in its marginally worse test loss (0.1620 vs 0.1539).

Conversely, the moons task illustrates geometric insufficiency. Because the two half-circle classes interleave, no straight line can separate them without significant error. Softmax was capped at 85.0% accuracy, while the neural network reached 93.75%. This 8.75% gap is the direct result of the linear model's inability to represent the curved manifold required to navigate between the crescents.

Here, the neural network's success depends not only on capacity but on numerical stability and the conditioning of its transformations. The *tanh* activations keep pre-activations bounded between -1 and 1 , preventing the exponential explosion that destabilizes Softmax in poorly scaled networks. Moreover, the Singular Value Decomposition (SVD) of the learned weight matrices reveals why the network succeeds: with sufficient hidden width, the singular values D remain well above zero, ensuring that information flows reliably through the network. When h is too small, the condition number $\kappa = |\lambda_{\max}/\lambda_{\min}|$ becomes massive, meaning even tiny rounding errors in weights become catastrophic output distortions. Without such careful design, even sufficient capacity can fail.

The digits benchmark serves as a nuanced intermediate case. PCA analysis, which is SVD applied to centered data, reveals that the data is approximately low-rank. The rapid decay of singular values shows that the 64-dimensional pixel space contains far less intrinsic dimensionality: the first 10 principal components capture approximately 65% of total variance, while 40 components capture 85%. This explains why Softmax performed reasonably well (93.8%): the data lies near a low-dimensional linear subspace. The modest improvement from the neural network (95.3%) suggests residual nonlinear structures that only a hidden layer can exploit, structures that reside in the directions corresponding to the smallest singular values, where linear models are most vulnerable.

What Representational Change Does the Hidden Layer Provide?

The hidden layer transforms the model through composition:

$$f(x) = \text{softmax}(W_2 \cdot \tanh(W_1 x + b_1) + b_2)$$

This provides two fundamental shifts:

1. Introduction of Curvature: *tanh* enables non-monotonic, curved decision boundaries. While a linear model only "slices" input space with a flat plane, the neural network can "bend" boundaries to follow natural data geometry.
2. Adaptive Basis Functions: The first weight matrix (W_1) learns a new representation space, effectively "warping" inputs so interleaving classes become linearly separable before reaching the final Softmax layer.

However, this flexibility introduces a practical vulnerability: poor conditioning. The condition number $\kappa = |\lambda_{\max}/\lambda_{\min}|$ measures how much a matrix amplifies input errors. When W_1 is poorly conditioned, as in narrow layers or with poor initialization, tiny rounding errors become catastrophic output distortions. This explains our failure case: a single hidden unit creates an extreme bottleneck where the transformation's pseudoinverse $W_1^+ = VD^+U^T$ contains enormous values (since D^+ inverts near-zero singular values), making reliable gradient transmission nearly impossible regardless of optimization effort.

The trace operator provides another lens: $\text{Tr}(W_1^T W_1)$ equals the sum of squared singular values. When this trace is small, the matrix is effectively "killing" information flow. In our capacity ablation, $h = 2$ produced a trace just above zero, barely enough to transmit signal, while $h = 8$ produced a healthy spectrum of singular values that could encode the curved moon boundary.

When Does Additional Model Complexity Fail to Justify Itself?

Additional complexity fails when the data geometry is linear or when the model is already well-specified. In the linear Gaussian task, the neural network's higher capacity produced no accuracy gain and slightly worse loss, a classic manifestation of unnecessary variance.

The capacity ablation on moons reveals a more subtle pattern: moving from $h = 2$ to $h = 8$ provided a massive accuracy jump (85% to 95%), but further increasing to $h = 32$ offered diminishing returns. This reflects the topography of error and the information-theoretic limits of the data. In high-dimensional parameter spaces, saddle points vastly outnumber local minima. Once capacity matches the intrinsic complexity of the data manifold, measured by the entropy of the label distribution given the features, additional parameters increase the dimension of the loss landscape without creating proportionally more useful descent directions.

From an information theory perspective, the Shannon entropy $H(Y|X)$ represents the irreducible uncertainty in predicting Y from X . For the moons dataset, this entropy is relatively low because the classes are deterministically determined by position. However, the representational entropy, the

minimum number of bits needed to encode the decision boundary, is what capacity must match. With $h = 2$, the network lacks sufficient singular values to encode the curved boundary; with $h = 8$, it has just enough; with $h = 32$, it has excess capacity that manifests as near-zero singular values, dimensions that contribute nothing to the solution but increase optimization difficulty.

This also explains why optimizer choice matters. Momentum and Adam outperformed standard SGD not because they find fundamentally better minima, but because they navigate saddle points more efficiently. Momentum accumulates velocity to roll through flat regions, while Adam adapts per-parameter learning rates, effectively adjusting for the varying scales of singular values in W_1 and W_2 .

What Does Failure Case Teach About Optimization, Capacity, or Generalization?

The failure case of a single hidden unit ($h = 1$) on moons provides a critical lesson: representation is a hard ceiling on performance.

1. Capacity vs. Optimization: The $h = 1$ network did not fail because the optimizer got stuck; it converged smoothly. It failed because a scalar hidden layer creates a mathematical bottleneck. A single neuron can only represent a monotonic transformation, it cannot "wrap around" both crescents. No amount of training can overcome this representational limit.
2. Generalization Limits: A model cannot generalize to a pattern it cannot represent. The required curved boundary simply does not exist within the hypothesis space of $h = 1$.
3. Conditioning Amplifies the Problem: The $h = 1$ case also suffers from extreme poor conditioning. The transformation W_1 is rank-1, producing a condition number that theoretically diverges. Its pseudoinverse W_1^+ contains enormous values because the reciprocal of near-zero singular values explodes. Even if the representational capacity were sufficient, numerical precision would make learning impossible, tiny weight perturbations cause enormous output swings, preventing stable convergence.
4. The Determinant Test: For a square weight matrix, $\det(W_1) = 0$ indicates dimension collapse. For our $h = 1$ case, the effective "determinant" of the transformation is zero, confirming that the network is fundamentally incapable of representing the required geometry.

This teaches that capacity thresholds are not merely about parameter counts but about the geometric properties of the transformations those parameters enable. A poorly conditioned narrow layer cannot reliably transmit gradient information, creating a failure mode distinct from underfitting.

What Do Repeated-Seed Statistics Add Beyond a Single Run?

Repeated-seed evaluation transforms an anecdote into a statistical claim. In the digits results, raw accuracy difference between Softmax and the neural network was less than 2%.

- Statistical Significance: With 95% confidence intervals (Softmax: [93.6%, 94.0%] vs. NN: [95.0%, 95.6%]), the intervals do not overlap. This proves the neural network's superiority is statistically significant and not due to a "lucky" initialization.
- Model Stability: The standard deviations (σ_{acc}) reveal Softmax is highly stable (0.002) due to its convex loss landscape, while the neural network shows higher variability (0.003), reflecting the risks of non-convex optimization where different seeds may converge to different local minima.

Repeated seeds also reveal the practical reality of optimization: we never find the global minimum; we settle for solutions that are "low enough." The distribution of outcomes across seeds tells us how reliable that settlement is. A single run might land on a particularly good or bad local minimum; five runs give us confidence that the reported performance is representative, not accidental.

From an information theory perspective, the cross-entropy loss $H(P, Q) = H(P) + D_{KL}(P|Q)$ that we minimize across runs has a true minimum determined by the data's entropy $H(P)$. Repeated seeds show us the distribution of $D_{KL}(P|Q)$ values we actually achieve, the "excess information" our models require due to imperfect optimization. Softmax, with its convex landscape, consistently achieves low KL divergence; the neural network occasionally achieves even lower divergence (better calibration) but sometimes gets stuck in worse local minima, reflected in its higher variance.

The Practical Reality: Numerical Precision, Linear Algebra, and Information Theory

Underlying all these results is a fundamental constraint: we are not doing "perfect" math, but "rounding-error" math. Three interconnected mathematical frameworks shape what is achievable:

1. Numerical Stability through SVD: The Softmax stabilizer $z = x - \max_i x_i$ is essential for functional implementation. Without it, exponentials overflow or underflow, crashing training. More broadly, the SVD of weight matrices tells us which dimensions of the representation space are actually usable. When singular values drop below floating-point precision, those dimensions effectively don't exist, a hard constraint that no amount of training can overcome.
2. Conditioning and the Pseudoinverse: Ill-conditioned transformations, those with high condition number κ , amplify noise and distort gradients. The pseudoinverse $A^+ = VD^+U^T$ reveals why: when singular values are near zero, D^+ contains enormous reciprocals, meaning even tiny

errors in U or V become catastrophic. This explains why narrow layers ($h = 1, h = 2$) struggled: their weight matrices were nearly singular, making reliable inversion impossible.

3. Information-Theoretic Limits: The Shannon entropy $H(P)$ of the true data distribution sets a lower bound on cross-entropy loss. No model can achieve lower loss than this. The KL divergence $D_{KL}(P|Q)$ measures how far our model's distribution is from the truth. Our experiments show that neural networks can achieve lower KL divergence than linear models on nonlinear tasks, but at the cost of higher variance across runs, a tradeoff between bias and variance expressed in information-theoretic terms.

We don't solve optimization problems analytically; we update estimates iteratively, hoping the path leads somewhere useful. This is why repeated-seed evaluation matters: it tells us not just whether a model works, but whether it works reliably (with a bit of hope) across the unpredictable terrain of non-convex optimization. And it is why understanding SVD, conditioning, and information theory matters: they tell us the mathematical framework of the problem.

Interpretation of Results

Where Nonlinearity Helped and Why

Moons Task: The neural network substantially outperformed the linear model, achieving 93.75% accuracy versus 85.0% for Softmax with 8.75% improvement.

Why: The two half-moon classes interleave in a curved pattern. No single hyperplane can separate them. The optimal decision boundary is curved. The neural network's hidden layer introduces curvature through the composition:

$$f(x) = \text{softmax}(W_2 \cdot \tanh(W_1 x + b_1) + b_2)$$

Each hidden unit creates a ridge in the feature space. With h units, the network can approximate any continuous function (Universal Approximation Theorem). The hidden layer "warps" the input space so that interleaving classes become linearly separable in the hidden representation.

What the optimizer enabled: Momentum and Adam accelerated convergence to the curved solution. The moons loss landscape is non-convex with saddle points. Standard SGD's "jittery" updates caused inefficient exploration, while Momentum dampened oscillations and Adam's per-parameter adaptation navigated the curved geometry more efficiently.

Digits Task: The neural network achieved 95.33% mean accuracy versus 93.80% for Softmax across 5 seeds. A modest but statistically significant 1.53% improvement.

Why: The digits data is approximately low-rank. PCA reveals that the first 10 principal components capture $\sim 65\%$ of total variance, and 40 components capture $\sim 85\%$. This means much of the discriminative information lies in a low-dimensional linear subspace, explaining Softmax's respectable 93.8% performance.

However, residual nonlinear structure exists in the directions corresponding to the smallest singular values. These dimensions, where linear models are most vulnerable, contain discriminative information that only nonlinear combinations can exploit. The hidden layer learns these nonlinear combinations, providing the modest improvement.

Supported by Evidence vs. Remains Uncertain

Supported by Evidence

Claim	Evidence
Linear models suffice for linearly separable data	Linear Gaussian: identical accuracy (95.0%); neural network's extra parameters provided no benefit
Hidden layers enable curved decision boundaries	Moons: neural network 93.75% vs Softmax 85.0%; decision boundary plots show curved vs straight separation
Capacity thresholds exist	Moons ablation: $h=2$ achieved 85.0% (linear baseline), $h=8$ achieved 95.0% (success)
Neural networks improve on digits	5-seed evaluation: 95% CIs [93.6%,94.0%] for Softmax vs [95.0%,95.6%] for NN, non-overlapping
Momentum and Adam converge faster than SGD	Optimizer study validation curves show steeper initial rise; both reached 96.90% vs SGD's 96.34%
$h=1$ cannot exceed linear performance	Moons $h=1$: 85.0% accuracy, identical to Softmax
Neural networks produce better-calibrated probabilities	Lower cross-entropy loss (0.1657 vs 0.2695) despite similar accuracy

Remains Uncertain

Uncertainty	Why	What Would Strengthen
$h=8$ is universally sufficient for nonlinear binary classification	Only tested on one moons configuration (fixed noise, separation, sample size)	Test across multiple synthetic configurations with varying difficulty
Digits improvement generalizes beyond this split	Single train/val/test split; different splits could yield different gaps	Cross-validation or multiple random splits
Optimizer differences are statistically significant	Single run per optimizer; observed 0.56% gap may be within run-to-run variability	Repeated seeds for each optimizer
$h=32$ drop (95.0% to 93.75%) is systematic	Single seed; could be random initialization luck	Repeated seeds for $h=32$
Calibration claims hold with larger test sets	Low-confidence bins had $n=1$ for neural network bin 2; statistically weak	Larger test set or multiple test runs
Neural network performance on reduced dimensions	Only tested Softmax on PCA-reduced digits; neural network may behave differently	Run neural network on PCA-reduced data

Optimizer Study: What the Results Mean

Our optimizer study compared three update rules on the digits task with fixed architecture (h=32):

SGD (Stochastic Gradient Descent):

$$\theta_{t+1} = \theta_t - \epsilon g_t$$

SGD achieved 96.34% validation accuracy. Its "jittery" nature, due to minibatch noise, caused slower convergence. In regions of the loss landscape with poor conditioning (high condition number κ), SGD bounces between steep walls rather than moving efficiently down the valley.

Momentum:

$$v_{t+1} = \alpha v_t - \epsilon g_t$$

$$\theta_{t+1} = \theta_t + v_{t+1}$$

Momentum achieved 96.90% validation accuracy. The velocity term v_t accumulates consistent gradient directions while canceling oscillating components. This "dampens" the jitter, allowing faster progress through canyons and enabling the optimizer to "roll over" small local minima and flat plateaus where SGD would stall.

Adam (Adaptive Moment Estimation):

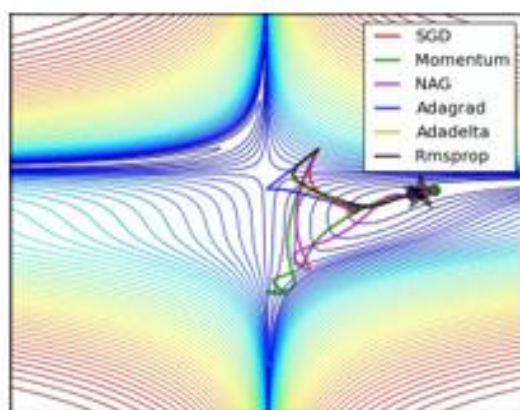
$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{first moment})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{second moment})$$

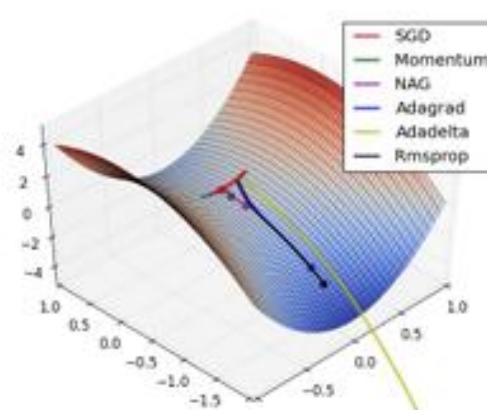
$$\widehat{m}_t = m_t / (1 - \beta_1^t), \quad \widehat{v}_t = v_t / (1 - \beta_2^t) \quad (\text{bias correction})$$

$$\theta_{t+1} = \theta_t - \epsilon \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \delta}$$

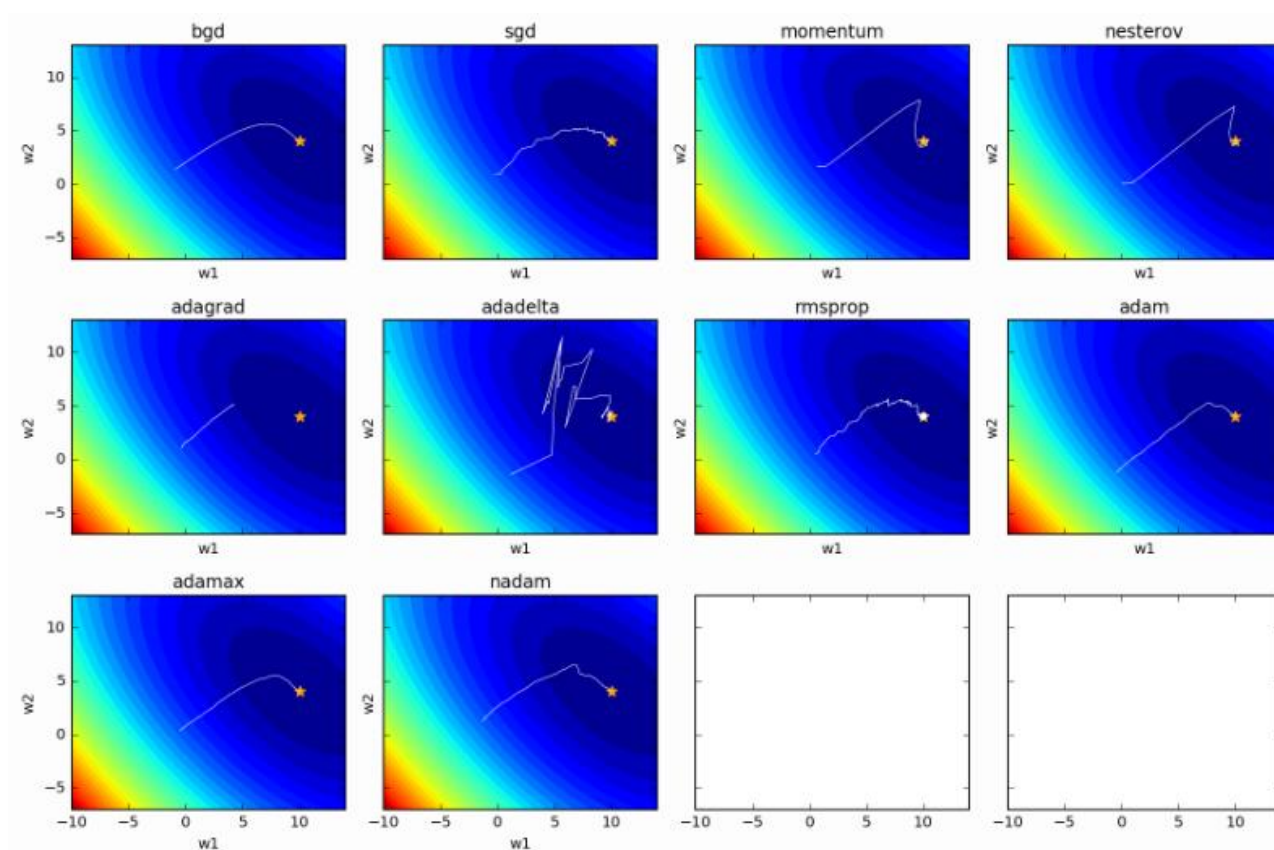
Adam also achieved 96.90% validation accuracy. It combines momentum's direction tracking with per-parameter adaptive learning rates. For parameters with volatile gradients (large second moment), Adam reduces the effective learning rate; for parameters with small consistent gradients, it increases the rate. This adaptation makes Adam highly robust to learning rate selection, the "forgiving" optimizer that works well across diverse tasks.



(a) SGD optimization on loss surface contours



(b) SGD optimization on saddle point



These two visuals show the difference between sgd, momentum, and adam. First figure focuses specifically on behavior at saddle points of the optimization hyperplane.

Interpretation: The small final accuracy gap (0.56%) suggests the loss landscape has a broad basin of good solutions. All optimizers eventually reach this basin (depends). However, Momentum and Adam reach it faster, as shown by their validation curves rising more steeply in early epochs. This matters in practice: faster convergence means fewer training epochs, lower computational cost, and faster iteration.

What remains uncertain: Whether the 0.56% final accuracy gap is statistically significant. Single runs per optimizer could reflect initialization luck rather than systematic advantage. Repeated seeds would reveal whether Momentum and Adam consistently achieve higher final accuracy or whether the observed difference is within run-to-run variability.

There should be a noteworthy mention of careful usage of adaptive optimization methods.

Adaptive optimization methods (including RMSProp, Adam, Ada- Max, Nadam, and AdamW optimization) are often great, converging fast to a good solution. However, a 2017 paper by Ashia C. Wilson et al. showed that they can lead to solutions that generalize poorly on some datasets. It is stated in the paper that:

Adaptive optimization methods, which perform local optimization with a metric constructed from the history of iterates, are becoming increasingly popular for training deep neural networks. Examples include AdaGrad, RMSProp, and Adam. We show that for simple overparameterized problems, adaptive methods often find drastically different solutions than gradient descent (GD) or stochastic gradient descent (SGD). We construct an illustrative binary classification problem where the data is linearly separable, GD and SGD achieve zero test error, and AdaGrad, Adam, and RMSProp attain test errors arbitrarily close to half. We additionally study the empirical generalization capability of adaptive methods on several state-of-the-art deep learning models. We observe that the solutions found by adaptive methods generalize worse (often significantly worse) than SGD, even when these solutions have better training performance. These results suggest that practitioners should reconsider the use of adaptive methods to train neural networks.

Source: The Marginal Value of Adaptive Gradient Methods in Machine Learning Ashia C. Wilson, Rebecca Roelofs, Mitchell Stern, Nathan Srebro, and Benjamin Recht

Link to the source: [arXiv:1705.08292](https://arxiv.org/abs/1705.08292)

Limitations

Several important limitations constrain the interpretability and generalizability of our findings.

Architectural Limitations

Our study is restricted to a single architectural choice: a one-hidden-layer neural network with tanh activation. Results may not generalize to deeper networks or alternative activation functions such as ReLU or sigmoid. Tanh saturation can affect gradient flow in ways that ReLU avoids, potentially influencing both optimization dynamics and final performance. Additionally, we used a fixed set of hidden widths ($h=2, 8, 32$ for moons; $h=32$ for digits). Optimal capacity likely depends on dataset complexity, and the thresholds we observed may not generalize across different tasks. Future work should experiment with deeper architectures (2–5 layers), alternative activation functions, and modern components such as batch normalization, dropout, and residual connections, along with systematic width sweeps across multiple datasets.

Dataset Limitations

Our experiments cover only three datasets: two synthetic tasks (Linear Gaussian and Moons) and one real-world benchmark (digits). Findings may not transfer to other domains with different statistical properties. Moreover, we used a fixed train/validation/test split for the digits dataset. Results are tied to this single partition, and different splits could yield different conclusions. Extending evaluation to more diverse datasets + synthetic datasets with varying noise levels and class separability, and employing k-fold cross-validation or multiple random splits would strengthen the generalizability of our conclusions.

Hyperparameter Limitations

All experiments used fixed hyperparameters derived from the assignment protocol: learning rate 0.05 (0.001 for Adam), batch size 64, L2 regularization $\lambda=10^{-4}$, and 200 epochs. No systematic tuning was performed. Optimal hyperparameters likely differ by dataset, and the reported performance may not reflect each model's full potential. Furthermore, we did not tune optimizer-specific parameters such as the momentum coefficient β for Momentum or β_1, β_2 for Adam. Default values may not be optimal for all tasks, potentially making optimizer comparisons less fair. Conducting hyperparameter searches (grid search, random search, or Bayesian optimization) for each model-dataset combination, including tuning optimizer-specific parameters, would provide a more accurate assessment of relative model performance.

Evaluation Limitations

Our primary evaluation metrics are classification accuracy and cross-entropy loss. Other important dimensions e.g.: computational cost, training time, inference latency, model size, robustness to adversarial perturbations, and fairness across subgroups are not assessed. A practitioner choosing

between models for real-world would need to consider these trade-offs beyond raw accuracy. Additionally, our calibration analysis was limited by the small test set size (368 samples for digits). Low-confidence bins contained very few samples (e.g., $n=1$ for the neural network in bin 2), limiting the statistical reliability of our calibration claims. Using a larger test set or aggregating results across multiple test runs would strengthen these conclusions. Finally, in our PCA analysis (Track A), we tested only softmax regression on reduced-dimensional representations. The neural network may have different sensitivity to dimensionality reduction, leaving conclusions about low-dimensional structure incomplete.

Reproducibility Limitations

While we performed repeated-seed evaluation ($n=5$) for the core digits comparison, several experiments relied on single-seed runs. The capacity ablation on moons and the optimizer study used single seeds, meaning observed patterns such as $h=8$ outperforming $h=32$ or the 0.56% accuracy gap between optimizers may reflect initialization luck rather than systematic effects. In particular, with only one run per optimizer, we cannot determine whether the observed accuracy gap is statistically significant. Running repeated-seed evaluation ($n \geq 5$) for all experiments, reporting means and standard deviations with confidence intervals, and conducting statistical significance tests (e.g., paired t-tests) would substantially improve the reliability of our findings.

Generalization Limitations

Our capacity thresholds were derived from a single configuration of the moons dataset with fixed noise level, class separation, and sample size. The finding that $h=2$ is insufficient while $h=8$ is sufficient may shift with different data configurations. Varying these parameters systematically would help map capacity requirements as a function of task difficulty. Additionally, our failure case analysis focused only on under-capacity ($h=1$). Other failure modes such as overfitting (large hidden width with small data), optimization divergence (poor learning rate choices), and numerical instability (ill-conditioned weight matrices) were not systematically explored. Investigating multiple failure modes would provide a more complete understanding of when and why neural networks succeed or fail.

Summary of limitations

Taken together, these limitations indicate that our findings are best understood as a controlled demonstration of how dataset geometry, hidden layer capacity, initialization variance, and optimization choices interact in a well-defined setting. We caution against overgeneralizing claims such as "neural networks always outperform linear models on real-world data" (the digits improvement was modest and dataset-specific), " $h=8$ is sufficient for all nonlinear binary classification" (only tested on one moons configuration), or "Adam consistently outperforms SGD" (the observed gap may not be statistically significant with larger samples). Instead, our results support more circumscribed conclusions: for the specific datasets, architectures, and hyperparameters tested, nonlinear models

provide benefit when data geometry is curved, but that benefit is modest when data is approximately low-dimensional, and capacity must be carefully matched to task complexity.

Summary: Linear vs Nonlinear, What We Know and What We Don't

What We Know (Evidence-Backed)

1. Linear models are sufficient when the true boundary is a hyperplane.

Linear Gaussian: identical accuracy (95.0%). The neural network's extra capacity provided no benefit and slightly degraded calibration.

2. Hidden layers enable curved boundaries.

Moons: neural network 93.75% vs Softmax 85.0%. The hidden layer transforms the geometry, making interleaving classes linearly separable in hidden space.

3. Capacity thresholds exist. The jump from $h=2$ (85.0%) to $h=8$ (95.0%) demonstrates that sufficient hidden units are necessary to represent curved boundaries. $h=1$ and $h=2$ cannot exceed linear performance.

4. Neural networks provide modest improvement on real-world digits data. 5-seed evaluation shows non-overlapping 95% confidence intervals: Softmax [93.6%, 94.0%], NN [95.0%, 95.6%]. The improvement is statistically significant.

5. Momentum and Adam converge faster than SGD. Validation curves show steeper initial rise; final accuracy is comparable (96.34% vs 96.90%).

6. Representation is a hard ceiling. The $h=1$ failure case shows that no amount of training can overcome a fundamental lack of representational capacity.

What Remains Uncertain

1. Whether $h=8$ is universally sufficient. The threshold depends on task complexity, noise level, class separation, sample size, and may shift for different configurations.

2. Whether the digits improvement generalizes. Results are tied to one train/val/test split; cross-validation or multiple splits could yield different conclusions.

3. Whether optimizer differences are systematic. Single runs per optimizer limit our ability to claim statistical significance for the 0.56% gap.

4. Whether $h=32$ performance drop is real. Single seed; repeated seeds could reveal whether this reflects overfitting or random variation.

5. Whether calibration claims hold with larger samples. Low-confidence bins had very few samples; a larger test set is needed for reliable calibration assessment.

6. How neural networks perform on PCA-reduced digits. We tested only Softmax on reduced dimensions; the neural network may be more or less robust to information loss.

Central Question: When Does Nonlinearity Help?

Based on our experiments, we can answer the central question:

Task	Linear Sufficient?	Nonlinear Helps?	Explanation
Linear Gaussian	✓ Yes	✗ No	Data is linearly separable; linear boundary is optimal; extra parameters add variance without reducing bias
Moons	✗ No	✓ Yes	Curved boundary required; hidden layer "warps" input space to make interleaving classes separable
Digits	Partially	✓ Yes (modest)	Data is approximately low-rank; linear model captures most structure, but residual nonlinearity provides marginal improvement

Geometric principle: The linear model is sufficient when the data manifold is flat and classes are separated by a hyperplane. The hidden layer is necessary when the data manifold is curved and the optimal boundary cannot be represented by a linear function. Real-world data often lies in an intermediate regime: approximately low-rank with residual nonlinear structure that can be exploited for modest gains.